

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH

VIỆN CÔNG NGHỆ THÔNG TIN, ĐIỆN VÀ ĐIỆN TỬ



**BÁO CÁO HỌC PHẦN
ĐIỆN TOÁN Đám Mây**

**TRIỂN KHAI WEBSITE SHOWROOM Ô TÔ
VINFAST TRÊN NỀN TẢNG
ĐIỆN TOÁN Đám Mây AWS**

Ngành: Công nghệ thông tin

Chuyên ngành: Công nghệ thông tin

Mã số: 7480201

Giảng viên hướng dẫn: TS. Hàn Trung Định

Sinh viên thực hiện: Nhóm 01

Huỳnh Nguyễn Quốc Bảo 22H1120002

Nguyễn Việt Dũng 22H1120004

Trần Đăng Nam 22H1120016

Phan Ngọc Thạch 22H1120024

Nguyễn Hoàng Việt 2251120129

Thành phố Hồ Chí Minh, tháng 01 năm 2026

**BÁO CÁO HỌC PHẦN
ĐIỆN TOÁN Đám MÂY**

**TRIỂN KHAI WEBSITE SHOWROOM Ô TÔ
VINFAST TRÊN NỀN TẢNG
ĐIỆN TOÁN Đám MÂY AWS**

Ngành: Công nghệ thông tin
Chuyên ngành: Công nghệ thông tin
Mã số: 7480201

Giảng viên hướng dẫn: TS. Hàn Trung Định

Sinh viên thực hiện: Nhóm 01

Huỳnh Nguyễn Quốc Bảo	22H1120002
Nguyễn Việt Dũng	22H1120004
Trần Đăng Nam	22H1120016
Phan Ngọc Thạch	22H1120024
Nguyễn Hoàng Việt	2251120129

Thành phố Hồ Chí Minh, tháng 01 năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng tiểu luận này là công trình nghiên cứu của nhóm. Các kết quả trình bày trong tiểu luận là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi sự hỗ trợ trong quá trình thực hiện tiểu luận đã được ghi nhận và cảm ơn. Tất cả các tài liệu tham khảo và trích dẫn trong tiểu luận đều đã được chỉ rõ nguồn gốc.

Chúng em xin chịu hoàn toàn trách nhiệm về nghiên cứu này.

TP.HCM, ngày 28 tháng 01 năm 2026

Nhóm 01

LỜI CẢM ƠN

Để hoàn thành tiểu luận này, nhóm đã nhận được sự hỗ trợ và hướng dẫn quý báu từ nhiều cá nhân và tổ chức. Chúng tôi xin gửi lời cảm ơn chân thành tới:

- TS. Hàn Trung Định, giảng viên hướng dẫn, đã tận tình chỉ bảo, định hướng cho nhóm trong quá trình thực hiện đề tài.
- Các Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Giao thông Vận tải TP.HCM, đã cung cấp nền tảng kiến thức vững chắc và tạo điều kiện thuận lợi cho nhóm trong quá trình học tập và nghiên cứu.
- Gia đình và bạn bè đã luôn động viên, khích lệ và đồng hành cùng nhóm trong suốt quá trình thực hiện tiểu luận.

Mặc dù đã nỗ lực hết mình, tiểu luận này không thể tránh khỏi những thiếu sót. Nhóm chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô và các bạn đọc để tiểu luận được hoàn thiện hơn.

Xin trân trọng cảm ơn!

TP.HCM, ngày 28 tháng 01 năm 2026

Nhóm 01

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
DANH MỤC CÁC TỪ VIẾT TẮT	vi
DANH MỤC BẢNG	vii
DANH MỤC HÌNH ẢNH	viii
MỞ ĐẦU	1
1. Lý do chọn đề tài	1
2. Mục tiêu nghiên cứu	1
3. Phạm vi & Đối tượng nghiên cứu.....	2
4. Phương pháp nghiên cứu	3
5. Kết cấu của tiểu luận.....	4
NỘI DUNG.....	5
CHƯƠNG 1. TỔNG QUAN VỀ DỰ ÁN VÀ CÔNG NGHỆ SỬ DỤNG.....	5
1.1. Tổng quan website showroom ô tô Vinfast	5
1.2. Kiến trúc tổng thể của ứng dụng trước triển khai.....	6
1.3. Công nghệ và dịch vụ sử dụng.....	7
1.3.1. Công nghệ xây dựng ứng dụng	7
1.3.2. Công cụ Docker và Docker Compose.....	7
1.3.3. Nền tảng đám mây AWS	9
1.3.4. Git/GitHub.....	16
CHƯƠNG 2. MÔ HÌNH TRIỂN KHAI HỆ THỐNG TRÊN ĐÁM MÂY	17
2.1. Mô hình triển khai trên AWS	17
2.2. Thiết kế dữ liệu MongoDB.....	19
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG TRÊN AMAZON WEB SERVICES	21
3.1. Khởi tạo và cấu hình hạ tầng Cloud trên AWS	21
3.1.1. Lựa chọn và khởi tạo EC2 Instance	21
3.1.2. Cấu hình Security Group để kiểm soát truy cập	22
3.1.3. Thiết lập IAM Role và Policy để kết nối dịch vụ	24
3.1.4. Cấu hình AWS Systems Manager (SSM) cho quản trị EC2.....	24

3.2. Chuẩn bị môi trường triển khai: Docker, Docker Compose và mã nguồn ..25	
3.2.1. Cài đặt và cấu hình Docker Engine.....26	
3.2.2. Cài đặt Docker Compose để quản lý multi-container.....27	
3.2.3. Clone mã nguồn và cấu trúc project.....28	
3.3. Cấu hình hệ thống và tích hợp Cloud Services.....29	
3.3.1. Cấu hình Backend thông qua biến môi trường (Environment Variables)29	
3.3.2. Tích hợp MongoDB Atlas - Database as a Service31	
3.3.3. Khởi tạo và tích hợp Amazon S3 - Object Storage.....32	
3.3.4. Thiết lập giám sát hệ thống với Amazon CloudWatch.....32	
3.4. Build và triển khai hệ thống với Docker Compose33	
3.4.1. Phân tích file docker-compose.yml.....33	
3.4.2. Dockerfile của Backend34	
3.4.3. Dockerfile của Frontend và vai trò của Nginx.....35	
3.4.4. Quy trình build và deploy hệ thống36	
3.5. Kiểm thử và xác minh hệ thống sau triển khai37	
3.5.1. Kiểm tra khả năng truy cập và giao diện người dùng.....37	
3.5.2. Kiểm tra chức năng xác thực và phân quyền quản lý38	
3.5.3. Kiểm tra chức năng quản lý xe (CRUD).....38	
3.5.4. Kiểm tra tính ổn định và khả năng phục hồi.....39	
3.5.5. Kết quả kiểm thử tổng hợp.....39	

CHƯƠNG 4. VẬN HÀNH, BẢO MẬT & KHẢ NĂNG MỞ RỘNG HỆ THỐNG

.....40	
4.1. Cơ chế vận hành hệ thống.....40	
4.2. Cập nhật & quản lý phiên bản40	
4.3. Quản lý cấu hình.....40	
4.4. Giám sát hệ thống41	
4.5. Khôi phục.....42	
4.6. Bảo mật.....42	

CHƯƠNG 5. ĐÁNH GIÁ KẾT QUẢ ĐẠT ĐƯỢC, HƯỚNG PHÁT TRIỂN	44
5.1. Đánh giá kết quả đạt được	44
5.2. Ưu điểm	44
5.3. Hạn chế	45
5.4. Hướng phát triển	45
KẾT LUẬN	47
PHỤ LỤC	49

DANH MỤC CÁC TỪ VIẾT TẮT

STT	Ký hiệu chữ viết tắt	Chữ viết đầy đủ
1	AI	Artificial Intelligence – Trí tuệ nhân tạo
2	LLM	Large Language Model - Mô hình ngôn ngữ lớn
3	JSON	JavaScript Object Notation
4	JWT	JSON Web Token
5	AWS	Amazon Web Services
6	EC2	Elastic Compute Cloud
7	SPA	Single Page Application
8	ODM	Object Data Modeling
9	AMI	Amazon Machine Image
10	EIP	Elastic IP
11	SSH	Secure Shell
12	HTTP	HyperText Transfer Protocol
13	SSD	Solid State Drive
14	LTS	Long Term Support
15	DBaaS	Database as a Service
16	CDN	Content Delivery Network
17	CRUD	Create, Read, Update, Delete
18	CI/CD	Continuous Integration/ Continuous Deployment
19	VPC	Virtual Private Cloud
20	EBS	Elastic Block Store

DANH MỤC BẢNG

Bảng 3.1. Các biến môi trường.....	30
------------------------------------	----

DANH MỤC HÌNH ẢNH

Hình 1.1. Kiến trúc Container hóa.....	8
Hình 1.2. AWS	10
Hình 1.3. Amazon EC2	11
Hình 1.4. Instance EC2 kết nối thông qua cơ chế Keypair	12
Hình 1.5. Triển khai dự án.....	13
Hình 1.6. Amazon S3	14
Hình 1.7. Amazon CloudWatch dùng để giám sát thông số của tất cả service.....	15
Hình 3.1. EC2 instance được khởi tạo và đang chạy trên AWS với instance type m7i-flex.large	21
Hình 3.2. Cấu hình Security Group cho phép SSH và truy cập HTTP/API.....	22
Hình 3.3. Kết nối SSH thành công vào EC2 instance	25
Hình 3.4. Docker và Docker Compose được cài đặt thành công trên EC2	26
Hình 3.5. Mã nguồn dự án được clone từ GitHub về EC2 instance.....	28
Hình 3.6. Cấu trúc project	28
Hình 3.7. Quá trình build Docker images cho frontend và backend	36
Hình 3.8. Các container được khởi chạy bằng Docker Compose	36
Hình 3.9. Trạng thái các container sau khi triển khai.....	37
Hình 3.10. Giao diện hệ thống quản lý xe.....	37
Hình 3.11. Giao diện quản lý xe sau khi admin đăng nhập.....	38

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh cuộc cách mạng công nghiệp diễn ra mạnh mẽ, chuyển đổi số không chỉ tập trung vào việc xây dựng các ứng dụng phần mềm mà còn nhấn mạnh vai trò của hạ tầng triển khai và vận hành hệ thống. Điện toán đám mây ngày càng khẳng định vị trí quan trọng nhờ khả năng cung cấp hạ tầng linh hoạt, dễ mở rộng, độ ổn định cao và tối ưu chi phí so với mô hình triển khai truyền thống. Do đó, việc nghiên cứu và thực hành triển khai hệ thống trên nền tảng cloud trở thành yêu cầu cần thiết đối với sinh viên và người làm trong lĩnh vực công nghệ thông tin.

AWS (Amazon Web Services) là nền tảng điện toán đám mây hàng đầu hiện nay, được sử dụng rộng rãi trong các hệ thống thực tế của doanh nghiệp. Việc lựa chọn AWS cho phép nhóm tiếp cận trực tiếp với mô hình triển khai hạ tầng phổ biến, từ cấu hình máy chủ, triển khai ứng dụng đến quản lý và vận hành hệ thống trong môi trường cloud. Thông qua đó, nhóm có cơ hội hiểu rõ hơn về cách thức một website được đưa vào hoạt động trên hạ tầng điện toán đám mây thay vì chỉ dừng lại ở mức phát triển ứng dụng.

Từ những lý do trên, nhóm nghiên cứu lựa chọn đề tài “Triển khai website showroom ô tô VinFast trên nền tảng điện toán đám mây AWS” với mục tiêu áp dụng kiến thức đã học vào thực tiễn, đồng thời học hỏi và rèn luyện kỹ năng triển khai, quản trị và vận hành website trên hạ tầng cloud. Đề tài không chỉ mang tính ứng dụng mà còn giúp nhóm nâng cao hiểu biết về quy trình triển khai hệ thống web trong bối cảnh công nghệ hiện đại hiện nay.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng và triển khai hoàn chỉnh ứng dụng web quản lý showroom VinFast trên nền tảng điện toán đám mây AWS. Hệ thống được phát triển với Frontend ReactJS và Backend Node.js/Express, đảm bảo thực hiện tốt các nghiệp vụ cốt lõi như quản lý danh mục xe, khách hàng và xử lý đặt cọc, đồng thời đáp ứng các yêu cầu khắt khe về độ ổn định và khả năng mở rộng trong môi trường thực tế.

Về mặt hạ tầng kỹ thuật, nhóm tập trung container hóa toàn bộ hệ thống bằng Docker và triển khai trên máy chủ ảo Amazon EC2 kết hợp với Nginx. Điểm trọng tâm của nghiên cứu là việc tối ưu hóa hệ thống thông qua hệ sinh thái dịch vụ AWS: tích

hợp Amazon S3 để lưu trữ hình ảnh nhằm giảm tải cho server; sử dụng Amazon CloudWatch để giám sát các chỉ số hiệu năng theo thời gian thực; và ứng dụng AWS Systems Manager để quản lý vận hành an toàn. Đặc biệt, vấn đề bảo mật được giải quyết triệt để bằng IAM Role, giúp kiểm soát quyền truy cập giữa các dịch vụ mà không cần lưu trữ khóa bí mật trong mã nguồn.

Bên cạnh đó, đề tài cũng chú trọng đảm bảo an toàn dữ liệu và quy trình vận hành thông qua việc tích hợp cơ sở dữ liệu MongoDB Atlas và cơ chế xác thực người dùng JWT. Các quy trình tự động hóa CI/CD cơ bản cũng được thiết lập, giúp nhóm nghiên cứu không chỉ tạo ra một sản phẩm có tính ứng dụng cao mà còn nắm vững kỹ năng triển khai và làm chủ hệ thống trên nền tảng đám mây hiện đại.

3. Phạm vi & Đối tượng nghiên cứu

Về phạm vi nghiên cứu, đề tài tập trung nghiên cứu và triển khai một ứng dụng web quản lý showroom ô tô VinFast theo kiến trúc ba lớp (frontend, backend, database), không mở rộng sang ứng dụng di động. Hệ thống sử dụng ReactJS cho frontend, Node.js/Express cho backend và MongoDB cho cơ sở dữ liệu, được triển khai trên nền tảng điện toán đám mây AWS với trọng tâm là dịch vụ EC2. Docker và Docker Compose được áp dụng để container hóa hệ thống, kết hợp Nginx làm reverse proxy, đồng thời tích hợp các dịch vụ cloud bên thứ ba MongoDB Atlas. Không chỉ dừng lại ở việc triển khai trên máy chủ ảo EC2, phạm vi nghiên cứu còn mở rộng sang việc tích hợp sâu hệ sinh thái AWS gồm: Amazon S3 để lưu trữ hình ảnh, CloudWatch để giám sát hệ thống, AWS Systems Manager hỗ trợ vận hành và cơ chế IAM Role để bảo mật kết nối. Đề tài giới hạn ở mức độ nghiên cứu kiến trúc hệ thống và quy trình triển khai cloud-native, chưa đi sâu vào các nghiệp vụ doanh nghiệp phức tạp.

Đối tượng nghiên cứu chính của đề tài là hệ thống quản lý và bán xe VinFast trực tuyến cùng quy trình triển khai ứng dụng web bằng Docker Compose trên hạ tầng AWS EC2. Bên cạnh đó, đề tài xem xét việc cấu hình và tích hợp các dịch vụ cloud liên quan phục vụ cho vận hành hệ thống, đồng thời hướng đến các nhóm người dùng gồm khách hàng, quản trị viên và nhà phát triển có nhu cầu tham khảo mô hình triển khai website trên nền tảng điện toán đám mây.

4. Phương pháp nghiên cứu

Để thực hiện đề tài, nhóm nghiên cứu áp dụng phương pháp nghiên cứu tài liệu nhằm xây dựng nền tảng lý thuyết và định hướng kỹ thuật cho hệ thống. Các tài liệu chuyên ngành liên quan đến điện toán đám mây AWS, Docker, ReactJS, Node.js và MongoDB được thu thập, phân tích và chọn lọc. Bên cạnh đó, nhóm tham khảo các tài liệu chính thức từ AWS và các công nghệ liên quan, cũng như một số dự án và mô hình triển khai thực tế tương tự, nhằm đảm bảo việc áp dụng đúng các chuẩn kỹ thuật và thông lệ tốt trong triển khai ứng dụng web trên cloud.

Trên cơ sở lý thuyết đã nghiên cứu, phương pháp mô hình hóa được sử dụng để thiết kế hệ thống. Nhóm tiến hành xây dựng kiến trúc tổng thể trên nền tảng AWS, xác định rõ vai trò của các thành phần Frontend, Backend, và Database. Trong mô hình này, Amazon S3 được tích hợp để đảm nhiệm vai trò lưu trữ đối tượng (hình ảnh xe), giúp tối ưu hóa dung lượng cho máy chủ ứng dụng. Đặc biệt, kiến trúc bảo mật được thiết kế chặt chẽ thông qua việc sử dụng IAM Role; đây không phải là một dịch vụ độc lập mà là cơ chế quản lý quyền truy cập với các Rule và Policy cụ thể, cho phép các thành phần trong hệ thống kết nối và tương tác với nhau một cách an toàn mà không cần lưu trữ khóa bí mật.

Song song với đó, phương pháp thực nghiệm được áp dụng mạnh mẽ trong quá trình phát triển và triển khai. Nhóm tiến hành xây dựng các module chức năng, container hóa toàn bộ ứng dụng bằng Docker và thử nghiệm cục bộ trước khi đưa lên AWS EC2. Quá trình triển khai thực tế không chỉ dừng lại ở việc khởi chạy máy chủ mà còn áp dụng AWS Systems Manager để hỗ trợ quản lý cấu hình và vận hành hệ thống một cách tự động hóa. Đồng thời, công tác theo dõi hoạt động được thực hiện thông qua Amazon CloudWatch, giúp giám sát liên tục các chỉ số (metrics) sức khỏe của EC2 và website để kịp thời xử lý các vấn đề phát sinh.

Cuối cùng, phương pháp đánh giá được sử dụng để kiểm tra mức độ đáp ứng của hệ thống so với mục tiêu đề ra. Hệ thống được đánh giá thông qua tính đầy đủ của chức năng, khả năng hoạt động ổn định và hiệu quả triển khai trên nền tảng cloud. Bên cạnh đó, quy trình phát triển phần mềm được thực hiện theo hướng lặp (iterative), có quản lý phiên bản mã nguồn, giúp nâng cao chất lượng sản phẩm và phản ánh đúng quy trình phát triển ứng dụng web hiện đại.

5. Kết cấu của tiểu luận

Ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, luận văn được cấu trúc thành 5 chương chính:

Chương 1: Tổng quan hệ thống và công nghệ sử dụng: Giới thiệu chức năng hệ thống, kiến trúc Client-Server và các công nghệ lõi như ReactJS, Node.js, Docker và nền tảng AWS.

Chương 2: Mô hình triển khai hệ thống trên đám mây: Thiết kế mô hình triển khai trên AWS & cấu trúc dữ liệu MongoDB.

Chương 3: Triển khai hệ thống trên AWS: Chi tiết các bước khởi tạo, cài đặt Docker, cấu hình biến môi trường, build ứng dụng và kiểm thử chức năng thực tế.

Chương 4: Vận hành, Bảo mật & Khả năng mở rộng hệ thống: Mô tả cơ chế tự động khởi động lại, quy trình cập nhật mã nguồn qua Git, giám sát hệ thống và các lớp bảo mật đa tầng.

Chương 5: Đánh giá kết quả và hướng phát triển: Tổng kết ưu điểm (triển khai thành công, ổn định), hạn chế (thiếu HTTPS, CI/CD) và đề xuất nâng cấp hệ thống trong tương lai.

NỘI DUNG

CHƯƠNG 1.

TỔNG QUAN VỀ DỰ ÁN VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan website showroom ô tô Vinfast

Hệ thống quản lý bán xe VinFast là một ứng dụng web được xây dựng phục vụ mục đích nghiên cứu và học tập, dựa trên việc tham khảo mô hình hoạt động và chức năng của các website bán xe trực tuyến hiện nay. Nội dung hệ thống sử dụng các dòng xe VinFast làm dữ liệu minh họa nhằm mô phỏng quy trình giới thiệu sản phẩm và quản lý nghiệp vụ trong lĩnh vực kinh doanh ô tô, không mang tính thương mại hay đại diện chính thức cho VinFast.

Về chức năng, hệ thống được thiết kế theo hai nhóm người dùng chính là khách hàng và quản trị viên, kế thừa các chức năng phổ biến từ các website showroom ô tô trực tuyến. Đối với khách hàng, hệ thống cho phép xem danh sách và thông tin chi tiết các dòng xe VinFast, thực hiện đặt cọc xe trực tuyến. Các chức năng này nhằm mô phỏng trải nghiệm người dùng cơ bản trên các website bán xe hiện nay.

Đối với quản trị viên, hệ thống cung cấp các chức năng quản lý dữ liệu và nghiệp vụ tương tự các hệ thống quản lý bán hàng trực tuyến, bao gồm quản lý danh sách xe, khách hàng và đơn đặt cọc, cùng với bảng điều khiển tổng quan hỗ trợ theo dõi tình trạng hoạt động của hệ thống. Các chức năng quản trị được xây dựng với mục tiêu minh họa quy trình vận hành và quản lý dữ liệu trong một hệ thống web thực tế.

Về mặt kỹ thuật, hệ thống được xây dựng theo kiến trúc client-server với frontend và backend tách biệt, nhằm đảm bảo tính linh hoạt, dễ mở rộng, thuận tiện cho việc triển khai trên nền tảng đám mây.

Trước khi triển khai hệ thống lên nền tảng điện toán đám mây, website được chạy và kiểm thử trong môi trường local nhằm đảm bảo các chức năng hoạt động ổn định. Trong giai đoạn này, frontend được triển khai tại địa chỉ <http://localhost:5173/>, đóng vai trò cung cấp giao diện người dùng cho website showroom, trong khi backend được cấu hình chạy trên port 80 để xử lý các yêu cầu API, đồng thời kết nối và làm việc với các dịch vụ cơ sở dữ liệu và lưu trữ, phục vụ cho toàn bộ hoạt động của hệ thống.

1.2. Kiến trúc tổng thể của ứng dụng trước triển khai

Ứng dụng web quản lý bán xe VinFast được thiết kế tách biệt rõ ràng giữa các thành phần giao diện, xử lý nghiệp vụ và lưu trữ dữ liệu. Kiến trúc này phù hợp với mô hình triển khai hiện đại trên nền tảng cloud computing, cho phép hệ thống hoạt động ổn định, dễ mở rộng và thuận tiện trong quá trình bảo trì cũng như nâng cấp.

Ở phía người dùng, ứng dụng được truy cập thông qua trình duyệt web như Chrome, Firefox, Safari hoặc Edge. Giao diện người dùng được xây dựng dưới dạng một Single Page Application (SPA) sử dụng ReactJS, đảm nhiệm việc hiển thị nội dung, tiếp nhận thao tác và gửi yêu cầu đến hệ thống xử lý trung tâm. React Router DOM được sử dụng để quản lý điều hướng giữa các trang như trang chủ, danh sách xe, chi tiết sản phẩm, đặt cọc và dashboard quản trị. Phần giao diện không trực tiếp xử lý nghiệp vụ mà chỉ đóng vai trò trung gian, giao tiếp với backend thông qua các RESTful API endpoints để lấy và hiển thị dữ liệu tương ứng. Vite được sử dụng làm build tool để tối ưu hóa quá trình phát triển và đóng gói ứng dụng.

Phần xử lý nghiệp vụ của hệ thống được triển khai trên máy chủ backend sử dụng Node.js và Express.js framework. Backend chịu trách nhiệm tiếp nhận các HTTP requests từ frontend, thực hiện các logic liên quan đến quản lý người dùng, xe VinFast, khách hàng, đặt cọc, giao dịch, kế toán và nhân sự, đồng thời xử lý xác thực thông qua JWT (JSON Web Token), phân quyền dựa trên vai trò người dùng (user/admin) và bảo mật dữ liệu. Backend cung cấp các RESTful API cho frontend sử dụng, đảm bảo sự tách biệt giữa giao diện và logic xử lý, giúp hệ thống có cấu trúc rõ ràng và dễ phát triển.

Dữ liệu của ứng dụng được lưu trữ trong hệ quản trị cơ sở dữ liệu MongoDB Atlas - một dịch vụ database-as-a-service trên cloud, hoạt động như một thành phần độc lập trong kiến trúc tổng thể. MongoDB được lựa chọn vì tính linh hoạt của cơ sở dữ liệu NoSQL, phù hợp với việc lưu trữ các document có cấu trúc đa dạng như thông tin xe, khách hàng, đơn đặt cọc và giao dịch. Backend sử dụng Mongoose ODM (Object Data Modeling) đóng vai trò trung gian trong việc truy cập và thao tác dữ liệu, đảm bảo tính nhất quán thông qua schema validation và an toàn thông tin. Việc sử dụng MongoDB Atlas trên cloud giúp hệ thống có khả năng tự động backup, replication và dễ dàng scale khi lưu lượng truy cập tăng cao.

1.3. Công nghệ và dịch vụ sử dụng

1.3.1. Công nghệ xây dựng ứng dụng

Frontend của hệ thống được xây dựng bằng ReactJS nhằm phát triển giao diện người dùng theo mô hình Single Page Application (SPA). ReactJS cho phép tổ chức giao diện theo hướng component, giúp dễ quản lý và tái sử dụng mã nguồn. Hệ thống sử dụng React Router DOM để quản lý điều hướng giữa các trang như trang chủ, danh sách xe, chi tiết xe và khu vực quản trị mà không cần tải lại trang. Vite được lựa chọn làm công cụ build và môi trường phát triển do tốc độ khởi động nhanh và hỗ trợ cập nhật nóng hiệu quả. Ngoài ra, Chart.js được tích hợp để hiển thị các biểu đồ thống kê trực quan trên dashboard quản trị, phục vụ việc theo dõi dữ liệu tổng quan.

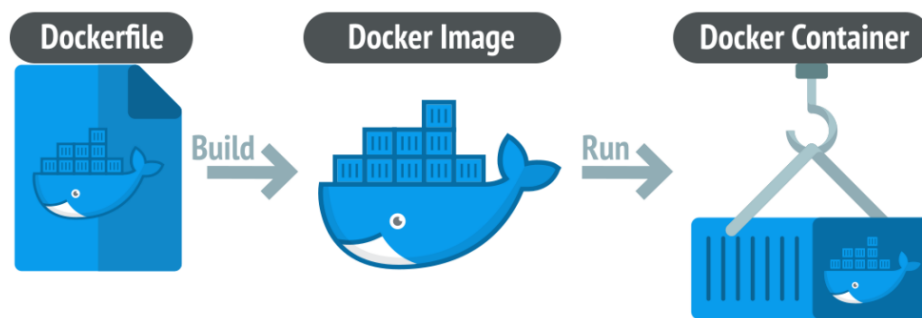
Backend của hệ thống được phát triển bằng Node.js kết hợp với Express.js, cung cấp các RESTful API phục vụ cho frontend. Backend chịu trách nhiệm xử lý logic nghiệp vụ, quản lý người dùng, xe, đặt cọc và các dữ liệu liên quan. Cơ sở dữ liệu MongoDB được sử dụng để lưu trữ dữ liệu, với Mongoose đóng vai trò trung gian giúp định nghĩa schema, kiểm soát dữ liệu và truy vấn một cách có cấu trúc. Hệ thống áp dụng cơ chế xác thực và phân quyền dựa trên JSON Web Token (JWT). Sau khi đăng nhập thành công, người dùng nhận được token để thực hiện các yêu cầu tiếp theo tới API. Phân quyền được triển khai theo vai trò người dùng và quản trị viên nhằm kiểm soát quyền truy cập vào các chức năng của hệ thống. Mật khẩu người dùng được mã hóa trước khi lưu trữ trong cơ sở dữ liệu để đảm bảo an toàn thông tin.

Việc giao tiếp giữa frontend và backend được thực hiện thông qua Axios, sử dụng các HTTP methods theo chuẩn RESTful. Các API được thiết kế rõ ràng và nhất quán, phục vụ cho các thao tác truy xuất và quản lý dữ liệu trong hệ thống. Đối với việc quản lý hình ảnh sản phẩm, hệ thống sử dụng Amazon S3 để lưu trữ ảnh, backend tích hợp AWS SDK for JavaScript v3 và Multer để xử lý upload và xóa ảnh. URL hình ảnh sau khi upload được lưu trong MongoDB và sử dụng để hiển thị trên website. Backend sử dụng middleware để xử lý upload ảnh và lưu trữ đường dẫn hình ảnh trong cơ sở dữ liệu phục vụ cho việc hiển thị trên website.

1.3.2. Công cụ Docker và Docker Compose

Để giải quyết bài toán triển khai frontend và backend trên cloud cùng với vấn đề đồng nhất giữa môi trường phát triển và môi trường sản phẩm, dự án sử dụng Docker

và Docker Compose để tiện lợi trong việc phát triển sản phẩm. Docker là một nền tảng containerization cho phép đóng gói ứng dụng cùng với toàn bộ dependencies, libraries và configuration files vào một đơn vị độc lập gọi là container. Khác với máy ảo truyền thống (Virtual Machine), container chia sẻ kernel của hệ điều hành host, do đó có kích thước nhẹ hơn, khởi động nhanh hơn và sử dụng tài nguyên hiệu quả hơn. Docker cho phép “đóng gói ứng dụng” cùng toàn bộ mã nguồn, thư viện và cấu hình vào một Dockerfile. Dockerfile sau quá trình Build sẽ tạo nên một Image, Image là thành phần để khởi tạo Container – nơi chạy ứng dụng đã được xây dựng từ Dockerfile trước đó. Container được chạy dựa trên 1 cơ chế ảo hóa của Docker, cho phép ứng dụng chạy bình thường trên nơi triển khai đám mây. Mỗi thành phần trong hệ thống của dự án này - mỗi Module - được vận hành trong một Container riêng biệt, đảm bảo tính cô lập (Isolation) và tránh bị xảy ra mâu thuẫn (Conflict) giữa các phiên bản.



Hình 1.1. Kiến trúc Container hóa

Nguồn: Internet

Trong dự án này, nhóm lựa chọn Docker để triển khai hệ thống vì các lý do sau:

- *Tính nhất quán môi trường*: Docker đảm bảo ứng dụng hoạt động giống nhau trên các môi trường development, staging và production. Điều này giúp loại bỏ vấn đề phổ biến “*It works on my machine*”, khi ứng dụng chạy tốt trên máy cá nhân nhưng gặp lỗi khi triển khai lên server.
- *Isolation (tính cô lập)*: Mỗi container hoạt động độc lập với các container khác. Khi một container gặp sự cố, các container còn lại vẫn tiếp tục hoạt động bình thường, góp phần tăng tính ổn định cho hệ thống và hỗ trợ việc debug dễ dàng hơn.
- *Portability (tính di động)*: Container có thể chạy trên bất kỳ hệ thống nào có cài Docker, từ máy cá nhân, server on-premise cho đến các nền tảng cloud

như AWS, Google Cloud hay Azure. Điều này giúp hệ thống dễ dàng di chuyển và triển khai trên nhiều hạ tầng khác nhau.

- *Scalability (khả năng mở rộng)*: Khi tải hệ thống tăng cao, có thể nhanh chóng tạo thêm các container để xử lý request và giảm số lượng container khi tải giảm. Quá trình này diễn ra trong thời gian ngắn, nhanh hơn đáng kể so với việc khởi tạo máy ảo mới.

Bên cạnh đó, Docker Compose có những lợi ích trong dự án như sau:

- *Đơn giản hóa quá trình triển khai*: Cho phép khởi động hoặc dừng toàn bộ hệ thống chỉ với một lệnh duy nhất, giúp quá trình deploy và rollback trở nên nhanh chóng và thuận tiện.
- *Quản lý sự phụ thuộc giữa các service*: Docker Compose đảm bảo các service được khởi động theo đúng thứ tự phụ thuộc, ví dụ backend được khởi động trước khi frontend gửi request.
- *Networking tự động*: Docker Compose tự động tạo một network nội bộ, cho phép các container giao tiếp với nhau thông qua tên service, mô phỏng cơ chế service discovery trong các hệ thống cloud.
- *Quản lý cấu hình và biến môi trường*: Các biến môi trường có thể được khai báo riêng cho từng service, giúp hệ thống linh hoạt khi thay đổi cấu hình giữa các môi trường khác nhau.
- *Mô phỏng kiến trúc microservices*: Mặc dù ở mức cơ bản, việc tách frontend và backend thành các service độc lập giúp nhóm tiếp cận với kiến trúc microservices, một mô hình phổ biến trong các hệ thống điện toán đám mây hiện đại.

1.3.3. Nền tảng đám mây AWS

a. Tổng quan về AWS

Điện toán đám mây (Cloud Computing) là công nghệ cung cấp tài nguyên tính toán từ một hệ thống máy tính lớn được đầu tư bởi công ty/tập đoàn, nhằm cung cấp các giải pháp tính toán, lưu trữ và triển khai ứng dụng thông qua Internet. Thay vì phải đầu tư và quản lý hạ tầng phần cứng vật lý, người dùng có thể thuê tài nguyên theo nhu cầu và chỉ trả phí cho những gì sử dụng. Mô hình này giúp doanh nghiệp và cá nhân giảm

chi phí đầu tư ban đầu (hạ tầng máy tính), dễ dàng mở rộng hoặc thu hẹp tài nguyên cho phép chi trả phí dựa theo nhu cầu sử dụng, cũng như dễ tiếp cận và triển khai nhanh.

Amazon Web Services (AWS) được lựa chọn làm môi trường triển khai chính. AWS cho phép người dùng triển khai hệ thống tương tự như một máy chủ vật lý, nhưng được vận hành trên hạ tầng đám mây với độ ổn định và khả năng mở rộng cao mà không phải phụ thuộc vào việc đầu tư chi phí phần cứng. AWS được lựa chọn bởi các lý do sau:

- AWS là một trong những nền tảng điện toán đám mây phổ biến và chiếm thị phần lớn trong thị trường toàn cầu. Việc lựa chọn này tạo điều kiện cho nhóm tiếp cận trực tiếp công nghệ đang được áp dụng trong doanh nghiệp hiện hành.
- Hỗ trợ chương trình Free Tier, giúp dự án được dùng một số dịch vụ cơ bản miễn phí trong thời gian giới hạn.
- Dễ dàng quản lý máy chủ bằng dịch vụ EC2, cho phép tạo và quản lý máy chủ ảo với cơ chế hoạt động tương đương một máy chủ vật lý.
- Tài liệu hướng dẫn và cộng đồng hỗ trợ của AWS lớn, giúp tiếp cận và triển khai dự án dễ dàng hơn.
- Từ nền tảng EC2 ban đầu, hệ thống có thể dễ dàng mở rộng sang các dịch vụ khác như lưu trữ, cơ sở dữ liệu, cân bằng tải hoặc bảo mật trong tương lai mà không cần thay đổi kiến trúc tổng thể.



Hình 1.2. AWS

Nguồn: Internet

b. Dịch vụ EC2 - Máy chủ ảo trong nền tảng AWS

Amazon EC2 (Elastic Compute Cloud) là dịch vụ cung cấp máy chủ ảo (virtual server) trong môi trường đám mây. EC2 cho phép người dùng tạo và quản lý các máy chủ có cấu hình phần cứng tùy chọn, tương tự như một máy tính vật lý. Thay vì đầu tư chi phí vào phần cứng máy tính từ đầu và cấu hình hệ điều hành máy chủ, yêu cầu khả năng quản trị hệ thống. EC2 đóng vai trò là máy chủ vật lý, chỉ cần cấu hình thông qua lệnh từ máy local, và kết nối thông qua giao thức SSH và ta dễ dàng quản lý một server mà không cần quan tâm đến bảo trì và đầu tư chi phí phần cứng. Hơn nữa, AWS cung cấp tính năng backup và khôi phục tự động, đảm bảo tính sẵn sàng cao cho hệ thống, cũng như EC2 cho phép mở rộng quy mô dễ dàng khi số lượng người dùng tăng lên. Vì lý do đó, EC2 được sử dụng để:

- Triển khai các ứng dụng web và dịch vụ API.
- Triển khai các phần mềm, hệ thống, ứng dụng.
- Lưu trữ và xử lý dữ liệu.
- Dùng các tác vụ tính toán, huấn luyện, chạy mô hình yêu cầu nhiều tài nguyên.



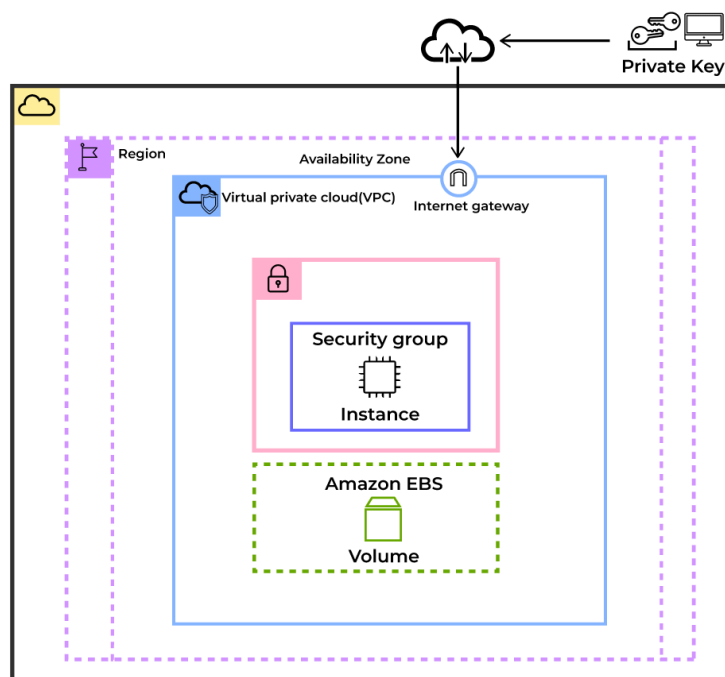
Hình 1.3. Amazon EC2

Nguồn: Internet

Để triển khai ứng dụng, ta cần phải khởi tạo (Launch) một đối tượng (Instance), các thành phần cần thiết để khởi tạo EC2 trên Cloud AWS bao gồm:

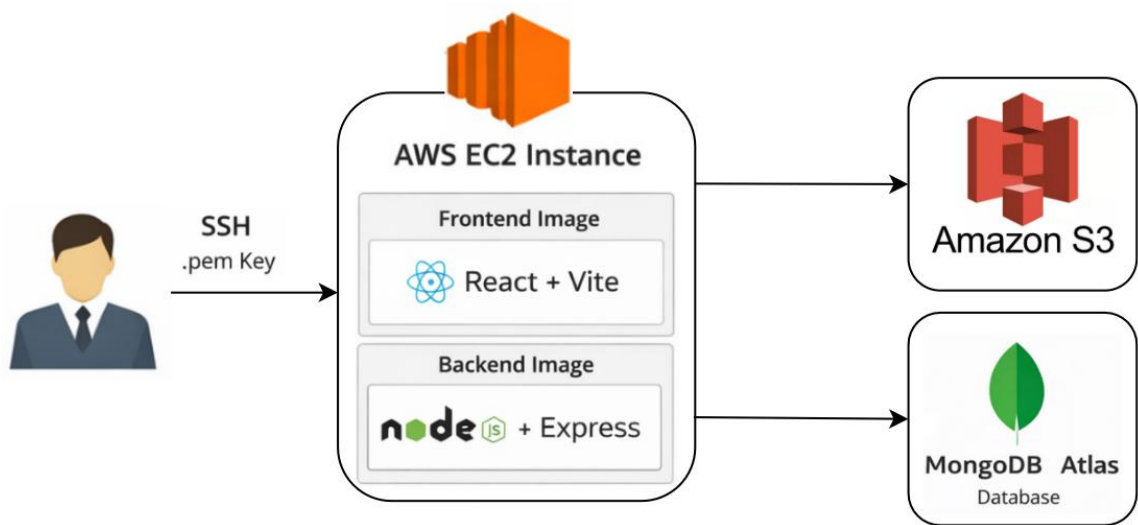
- *Amazon Machine Image (AMI)*: là hệ điều hành dùng để tạo EC2 instance . Thông thường sẽ là hệ điều hành Ubuntu Server 22.04.
- *Instance Type (Cấu hình phần cứng)*: xác định tài nguyên phần cứng của EC2, bao gồm số lượng CPU, dung lượng RAM hay GPU, thông lượng mạng,.. Việc lựa chọn instance type phù hợp giúp tối ưu chi phí và hiệu suất cho hệ thống triển khai, trong dự án này sử dụng m7i.

- *Key Pair*: là cặp khóa public key – được lưu trên EC2 và private key – file .pem do người dùng lưu trữ, được sử dụng để truy cập EC2 thông qua giao thức SSH.
- *Elastic IP (EIP)*: cho phép cấu hình một địa chỉ IP tĩnh cho EC2, phù hợp với các hệ thống triển khai thực tế và môi trường sản xuất. Tránh trường hợp IP thay đổi sau mỗi lần stop và start EC2.



Hình 1.4. Instance EC2 kết nối thông qua cơ chế Keypair

Nguồn: Internet



Hình 1.5. Triển khai dự án

c. Dịch vụ lưu trữ đối tượng Amazon S3 (Object Storage)

Amazon S3 (Simple Storage Service) là dịch vụ Object Storage do AWS cung cấp, cho phép lưu trữ và truy xuất dữ liệu với độ bền cao, tính sẵn sàng lớn và khả năng mở rộng gần như không giới hạn. Dữ liệu trong Amazon S3 được tổ chức theo cấu trúc bucket và object, trong đó mỗi object đại diện cho một tệp dữ liệu cụ thể và bucket là nơi chứa các object.

Object Storage là mô hình lưu trữ các dữ liệu dạng hình ảnh, video và các tệp đa phương tiện. Trong đó dữ liệu được quản lý dưới dạng các đối tượng (object), mỗi đối tượng bao gồm ba thành phần chính: dữ liệu, metadata và một định danh duy nhất. Trong phạm vi dự án này, Amazon S3 được sử dụng để lưu trữ hình ảnh.

Amazon S3 mang lại nhiều ưu điểm quan trọng cho dự án triển khai trên nền tảng AWS, cụ thể:

- Tách biệt giữa lưu trữ và xử lý dữ liệu: Amazon S3 cho phép lưu trữ dữ liệu độc lập với máy ảo EC2, giúp giảm tải cho EC2 lưu trữ các object và tăng tính linh hoạt trong quản lý hệ thống.
- Mở rộng linh hoạt: Dịch vụ cung cấp dung lượng lưu trữ gần như không giới hạn, đáp ứng hiệu quả sự gia tăng về số lượng dữ liệu mà không cần nâng cấp hạ tầng phần cứng.
- An toàn dữ liệu và độ tin cậy cao: Dữ liệu được sao chép trên nhiều hệ thống lưu trữ vật lý, đảm bảo an toàn và toàn vẹn trước các sự cố phần cứng.

- Thuộc hệ sinh thái AWS: Amazon S3 tích hợp trực tiếp với EC2 thông qua IAM Role, giúp quản lý quyền truy cập an toàn đến dịch vụ lưu trữ và tương tác mượt mà giữa các dịch vụ thuộc AWS.
- Hiệu suất truy cập: Dữ liệu trên S3 có thể được truy cập qua HTTP/HTTPS và kết hợp với dịch vụ CloudFront (CDN) của AWS để giảm độ trễ khi hiển thị nội dung.



Hình 1.6. Amazon S3

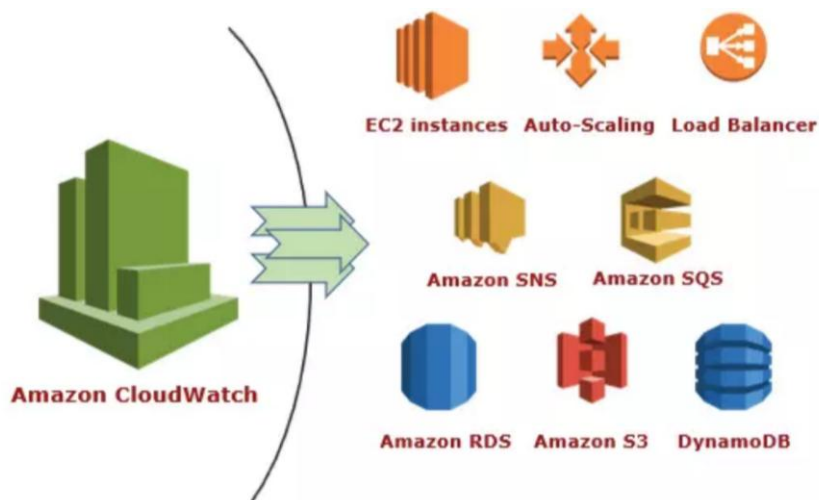
Nguồn: Internet

d. Dịch vụ giám sát Amazon CloudWatch

Amazon CloudWatch là dịch vụ giám sát và quan sát (monitoring và observability) do AWS cung cấp, cho phép thu thập, lưu trữ và phân tích các chỉ số (metrics), nhật ký hệ thống (logs) và sự kiện (events) từ các tài nguyên và ứng dụng chạy trên nền tảng AWS. CloudWatch giúp người quản trị theo dõi trạng thái hoạt động của hệ thống theo thời gian thực, từ đó đánh giá hiệu năng và phát hiện sớm các sự cố.

Amazon CloudWatch được sử dụng để giám sát hoạt động của các máy ảo EC2, dịch vụ lưu trữ Amazon S3 và các thành phần backend của ứng dụng. Thông qua CloudWatch, hệ thống có thể theo dõi hiệu năng của EC2 bằng các chỉ số quan trọng như mức sử dụng CPU, bộ nhớ, lưu lượng mạng và trạng thái dịch vụ.

Ngoài ra, CloudWatch Logs cho phép thu thập và phân tích log của ứng dụng backend, hỗ trợ việc kiểm tra lỗi và tối ưu hiệu năng. Bên cạnh đó, dịch vụ còn cho phép thiết lập cảnh báo (Alarm) nhằm tự động thông báo hoặc kích hoạt hành động khi hệ thống vượt quá ngưỡng cho phép, góp phần nâng cao độ ổn định và độ tin cậy của hệ thống.



Hình 1.7. Amazon CloudWatch dùng để giám sát thông số của tất cả service
e. Dịch vụ AWS Systems Manager

AWS Systems Manager là dịch vụ quản lý và vận hành hạ tầng cho phép người quản trị giám sát, cấu hình và điều khiển các tài nguyên AWS, đặc biệt là các máy ảo EC2, một cách tập trung và an toàn thông qua giao diện quản lý của AWS hoặc dòng lệnh (CLI). Trong dự án này, AWS Systems Manager được sử dụng để quản lý các máy ảo EC2. Thay vì phải kết nối trực tiếp tới từng máy ảo thông qua SSH và private key, Systems Manager cho phép người dùng truy cập và thực thi lệnh trên EC2 trực tiếp từ giao diện AWS Console thông qua tính năng Session Manager mà không cần kết nối SSH. Cơ chế này giúp giảm thiểu rủi ro bảo mật liên quan đến việc quản lý khóa truy cập (ví dụ: mất private key, private key bị sao chép...) và tăng tính an toàn cho hệ thống.

Bên cạnh đó, AWS Systems Manager hỗ trợ tự động hóa các tác vụ vận hành như triển khai ứng dụng, cập nhật cấu hình và quản lý trạng thái máy ảo trên nhiều EC2 cùng lúc. Nhờ việc tích hợp với dịch vụ IAM, cho phép rằng chỉ những máy ảo và người dùng được cấp quyền phù hợp mới có thể thực hiện các thao tác quản lý, từ đó nâng cao khả năng kiểm soát và bảo mật trong quá trình triển khai và vận hành hệ thống.

f. Dịch vụ AWS IAM (Identity and Access Management)

AWS Identity and Access Management (IAM) là một dịch vụ bảo mật tổng thể của AWS, chịu trách nhiệm quản lý danh tính (Identity) và quyền truy cập đối với tất cả các tài nguyên trong tài khoản AWS. IAM không trực tiếp cấp quyền, mà cung cấp cơ chế để tạo và quản lý các thực thể (IMA Role - Identity Entity) và quy tắc phân quyền

(IMA Policy). Điều này giúp đảm bảo rằng chỉ những thực thể được cấp quyền phù hợp mới có thể truy cập và sử dụng các dịch vụ trong hệ thống.

Trong dự án, nhóm sử dụng dịch vụ AWS IAM để tạo một IAM Role – là Identity Entity và gắn trực tiếp vào máy ảo EC2, thay vì sử dụng AWS Access Key và Secret Key được hardcode trong mã nguồn ứng dụng. Việc sử dụng IAM Role giúp EC2 tự động nhận quyền truy cập cần thiết thông qua cơ chế xác thực nội bộ của AWS, ví dụ truy cập trực tiếp đến service S3 mà không yêu cầu cung cấp cặp khóa để truy cập. Với dự án lần này, IAM Role trong dự án được gán các phân quyền - policy sau:

- *AmazonS3FullAccess*: Cho phép máy ảo EC2 truy cập và thao tác với dịch vụ Amazon S3, phục vụ việc lưu trữ và truy xuất hình ảnh của ứng dụng.
- *AmazonSSMMangedInstanceCore*: Cho phép EC2 được quản lý thông qua AWS Systems Manager, hỗ trợ truy cập và vận hành máy ảo trên AWS Console mà không cần sử dụng SSH và private key.
- *CloudWatchAgentServerPolicy*: Cho phép EC2 gửi log và các chỉ số giám sát lên Amazon CloudWatch, phục vụ cho việc theo dõi hiệu năng và trạng thái hệ thống.

1.3.4. Git/GitHub

Hệ thống tận dụng Git và GitHub làm nền tảng quản lý phiên bản và hỗ trợ cộng tác nhóm, giúp theo dõi chi tiết lịch sử phát triển, thực hiện rollback nhanh chóng và triển khai quy trình CI/CD hiệu quả trên môi trường container hoặc cloud. Việc áp dụng bộ công cụ này không chỉ đảm bảo quản lý phiên bản chặt chẽ và sao lưu mã nguồn tập trung nhằm giảm thiểu rủi ro mất dữ liệu, mà còn tối ưu hóa khả năng làm việc song song giữa các thành viên mà không gây xung đột mã nguồn lớn. Đồng thời, thông qua cơ chế Pull Request của GitHub, đội ngũ có thể thực hiện Code Review kỹ lưỡng trước khi tích hợp vào nhánh chính, tạo tiền đề vững chắc cho việc tự động hóa các khâu xây dựng và triển khai hệ thống trong tương lai.

CHƯƠNG 2.

MÔ HÌNH TRIỂN KHAI HỆ THỐNG TRÊN ĐÁM MÂY

2.1. Mô hình triển khai trên AWS

Hệ thống Showroom ô tô VinFast được triển khai trên nền tảng Amazon Web Services (AWS) sử dụng dịch vụ máy chủ ảo Amazon EC2 (Elastic Compute Cloud). Mô hình triển khai được thiết kế theo hướng "Containerization" (đóng gói ứng dụng) nhằm đảm bảo sự nhất quán giữa môi trường phát triển và môi trường vận hành thực tế. Docker Compose được sử dụng để quản lý và khởi chạy đồng thời các container trong hệ thống, bao gồm frontend và backend, giúp đơn giản hóa quá trình triển khai và vận hành.

Cấu hình hạ tầng triển khai:

- Máy chủ (Server): Sử dụng duy nhất một EC2 Instance chạy hệ điều hành Ubuntu Server. Đây là nơi lưu trữ mã nguồn, Docker Engine và thực thi các container của ứng dụng.
- Công cụ quản lý (Orchestrator): Sử dụng Docker Compose đóng vai trò là công cụ điều phối trung tâm. File cấu hình docker-compose.yml đóng vai trò như nhà điều phối (Orchestrator), giúp điều khiển các tác vụ có thể bắt đầu khởi động hoặc dừng các module cùng một lúc, thay vì phải kích hoạt từng module riêng lẻ. Bên cạnh đó nó cũng định nghĩa toàn bộ kiến trúc dịch vụ, mạng (network) và các biến môi trường cần thiết, cho phép khởi tạo toàn bộ hệ thống chỉ với một câu lệnh.
- Cơ chế kết nối và bảo mật (IAM Role): Để đảm bảo an toàn trong việc kết nối giữa các thành phần, hệ thống không lưu trữ khóa truy cập (Access Key) trong mã nguồn mà sử dụng cơ chế IAM Role. Đây là tập hợp các quy tắc (Rules) và chính sách (Policies) được gán trực tiếp cho EC2 Instance, ủy quyền cho máy chủ này được phép truy cập hợp lệ vào các dịch vụ AWS khác như S3 hay CloudWatch.
- Công cụ vận hành (AWS Systems Manager): Dịch vụ AWS Systems Manager được tích hợp để hỗ trợ quản lý cấu hình và vận hành máy chủ EC2, cho phép

thực hiện các tác vụ quản trị từ xa một cách an toàn và tự động hóa các quy trình bảo trì.

Hệ thống được chia thành hai container độc lập hoạt động song song trên cùng một EC2 instance:

- Frontend Container:
 - + Chứa mã nguồn React đã được build thành các file tĩnh (static files).
 - + Sử dụng Nginx làm Web Server để phục vụ các file giao diện này cho người dùng.
 - + Container này lắng nghe các yêu cầu từ internet thông qua Port 80 của EC2.
- Backend Container:
 - + Chạy môi trường Node.js và framework Express.js.
 - + Đảm nhiệm xử lý logic nghiệp vụ, xác thực người dùng (JWT) và kết nối với các dịch vụ bên thứ ba (Database, Cloud Storage).
 - + Container này hoạt động ở Port 5000 và giao tiếp với Frontend thông qua API.
 - + Tích hợp với Amazon S3: Thay vì lưu trữ hình ảnh xe trên ổ cứng cục bộ, backend kết nối với dịch vụ lưu trữ đối tượng Amazon S3. Điều này giúp tối ưu hóa dung lượng máy chủ, đảm bảo khả năng mở rộng lưu trữ không giới hạn và tăng tốc độ truy xuất hình ảnh.

Bên cạnh việc vận hành, hệ thống tích hợp Amazon CloudWatch để giám sát toàn diện sức khỏe của EC2 Instance. Dịch vụ này liên tục thu thập các chỉ số (metrics) quan trọng như mức sử dụng CPU, dung lượng RAM, và lưu lượng mạng, giúp quản trị viên nắm bắt trạng thái hệ thống theo thời gian thực.

AWS Security Groups được thiết lập như một tường lửa ảo để kiểm soát traffic vào/ra, chỉ mở các ports cần thiết như port 80 (HTTP) và port 22 (SSH). Elastic IP được gán cho instance để đảm bảo địa chỉ IP cố định. Việc kết hợp kiến trúc containerized với hệ sinh thái dịch vụ AWS (S3, CloudWatch, Systems Manager) tạo nên một hệ thống hiện đại, có tính sẵn sàng cao (High Availability), dễ dàng giám sát và mở rộng trong tương lai.

Luồng hoạt động đi từ tương tác của người dùng đến việc xử lý dữ liệu ở backend. Người dùng truy cập website qua trình duyệt bằng địa chỉ IP public của AWS EC2. Yêu cầu được tiếp nhận bởi container Nginx và trả về giao diện. Khi người dùng thực hiện thao tác (xem xe, thêm mới), frontend gửi HTTP request đến backend tại cổng 5000. Backend xử lý nghiệp vụ, truy vấn dữ liệu từ MongoDB Atlas và thực hiện upload/lấy hình ảnh từ Amazon S3 thông qua quyền truy cập của IAM Role, sau đó trả kết quả về cho frontend để cập nhật giao diện hiển thị.

2.2. Thiết kế dữ liệu MongoDB

Hệ thống sử dụng MongoDB Atlas, một dịch vụ cơ sở dữ liệu NoSQL trên nền tảng đám mây, mang lại khả năng mở rộng linh hoạt và hiệu năng cao. Thay vì lưu trữ dữ liệu dạng bảng như SQL truyền thống, dữ liệu được tổ chức thành các Documents (tài liệu JSON) và lưu trữ trong các Collections (Bộ sưu tập).

Cấu trúc dữ liệu chính bao gồm 03 collection quan trọng:

- Collection users (Người dùng & Quản trị viên): Lưu trữ thông tin xác thực và phân quyền người dùng.
 - + Các trường chính: username, password (đã mã hóa), email, role (admin/user), createdAt.
 - + Chức năng: Phục vụ cho quá trình đăng nhập, bảo mật JWT và kiểm soát quyền truy cập vào trang quản trị (Dashboard).
- Collection cars (Thông tin xe): Lưu trữ toàn bộ thông tin về các dòng xe VinFast đang kinh doanh.
 - + Các trường chính: modelName, price, description, specs (thông số kỹ thuật), imageUrl (liên kết đến ảnh trên Amazon S3), status (còn hàng/hết hàng).
 - + Chức năng: Hiển thị danh sách sản phẩm trên trang chủ và trang chi tiết xe.
- Collection orders (Đơn hàng/Yêu cầu tư vấn): Lưu trữ thông tin đăng ký mua xe hoặc yêu cầu tư vấn từ khách hàng.
 - + Các trường chính: customerName, phoneNumber, email, orderDate, status (pending/processed).

- + Quan hệ dữ liệu: Collection này thiết lập quan hệ với collection cars thông qua trường tham chiếu carId (kiểu dữ liệu ObjectId). Điều này giúp hệ thống biết được khách hàng đang quan tâm đến mẫu xe nào mà không cần lặp lại toàn bộ thông tin xe trong đơn hàng.

Cơ chế liên kết dữ liệu: Mặc dù là NoSQL, hệ thống vẫn duy trì tính toàn vẹn dữ liệu thông qua việc sử dụng ObjectId để tham chiếu giữa các collection (ví dụ: orders tham chiếu đến cars). Backend sử dụng thư viện Mongoose để quản lý các schema và thực hiện các thao tác "populate" (kết nối dữ liệu) khi cần truy vấn thông tin chi tiết.

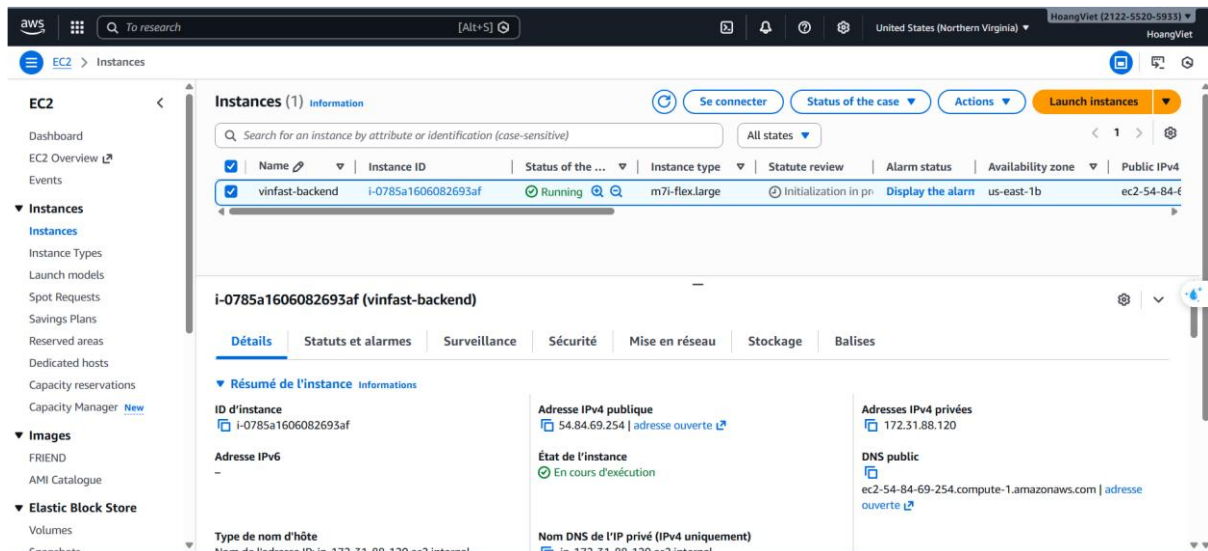
CHƯƠNG 3.

TRIỂN KHAI HỆ THỐNG TRÊN AMAZON WEB SERVICES

Chương này trình bày chi tiết quy trình triển khai hệ thống Car-Management lên nền tảng điện toán đám mây Amazon Web Services (AWS). Nội dung tập trung vào việc mô tả cách thức áp dụng các kiến thức lý thuyết đã học trong học phần Điện toán đám mây vào thực tiễn triển khai, bao gồm khởi tạo hạ tầng, cấu hình môi trường và đảm bảo hệ thống hoạt động ổn định với khả năng truy cập từ Internet.

3.1. Khởi tạo và cấu hình hạ tầng Cloud trên AWS

3.1.1. Lựa chọn và khởi tạo EC2 Instance



Hình 3.1. EC2 instance được khởi tạo và đang chạy trên AWS với instance type m7i-flex.large

Quá trình khởi tạo EC2 Instance được thực hiện thông qua AWS Management Console với các bước cụ thể. Đầu tiên, nhóm truy cập vào dịch vụ EC2 và chọn "Launch Instance" để bắt đầu tạo máy chủ mới. Sau đó nhóm cấu hình các thông số kỹ thuật phù hợp với quy mô và yêu cầu của đề án.

Thông số kỹ thuật EC2 Instance được lựa chọn:

- *Hệ điều hành*: Ubuntu Server 22.04 LTS (Long Term Support) - được chọn vì tính ổn định cao, hỗ trợ dài hạn từ Canonical, và có cộng đồng người dùng lớn giúp dễ dàng tìm kiếm giải pháp khi gặp vấn đề kỹ thuật.
- *Instance type*: m7i-flex.large với cấu hình gồm 2 vCPU và dung lượng RAM lớn, thuộc dòng M7i – general purpose instance của AWS, được thiết kế để

đáp ứng tốt các ứng dụng có nhu cầu xử lý ổn định và hiệu năng cao hơn so với các dòng burstable.

- *Dung lượng lưu trữ*: 20GB SSD (Solid State Drive) - dung lượng này đủ để chứa hệ điều hành, các công cụ cần thiết, mã nguồn ứng dụng và các log file. Việc sử dụng SSD thay vì HDD giúp tăng tốc độ đọc/ghi dữ liệu, cải thiện hiệu năng tổng thể của hệ thống.
- *Vùng triển khai (Region)*: Nhóm chọn region gần Việt Nam nhất để giảm độ trễ (latency) khi người dùng trong nước truy cập hệ thống.

Sau khi hoàn tất cấu hình, AWS tạo ra một instance mới và cấp cho nó một địa chỉ Public IPv4, đây chính là địa chỉ mà người dùng sẽ sử dụng để truy cập vào hệ thống từ Internet. Đồng thời, AWS cũng tạo ra một key pair (private key và public key) phục vụ cho việc xác thực khi SSH vào server, đảm bảo chỉ những người có private key mới có thể truy cập và quản trị server.

3.1.2. Cấu hình Security Group để kiểm soát truy cập

The screenshot displays the AWS Management Console interface. At the top, the 'Instances (1)' page is active, showing a table with one instance: 'vinfast-backend' (ID: i-0785a1606082693af), which is in a 'Running' state. Below this, the 'Security Groups' tab is selected for the instance, showing a table of rules. The table has columns for Name, security group rule ID, Ports beach, Protocol, Source, and Security groups. Four rules are listed, all with 'launch-wizard-1' as the security group.

Name	security group rule ID	Ports beach	Protocol	Source	Security groups
-	sgr-0850f613ca09eddf9	443	TCP	0.0.0.0/0	launch-wizard-1
-	sgr-09cf28f2a7cc6e5ab	80	TCP	0.0.0.0/0	launch-wizard-1
-	sgr-097bf526a9ebb3f5b	5000	TCP	0.0.0.0/0	launch-wizard-1
-	sgr-072589aad380bb2ef	22	TCP	0.0.0.0/0	launch-wizard-1

Hình 3.2. Cấu hình Security Group cho phép SSH và truy cập HTTP/API

Security Group đóng vai trò như một tường lửa ảo (virtual firewall) ở mức instance, kiểm soát luồng traffic đi vào (inbound) và đi ra (outbound) khỏi EC2 instance. Đây là một trong những lớp bảo mật quan trọng nhất trong kiến trúc AWS, giúp bảo vệ hệ thống khỏi các truy cập trái phép và tấn công từ bên ngoài.

Khi tạo EC2 instance, AWS tự động tạo một Security Group mặc định. Tuy nhiên, Security Group này thường chỉ mở port 22 cho SSH. Do đó, nhóm cần tùy chỉnh lại Security Group để phù hợp với yêu cầu của hệ thống Car-Management.

Cấu hình chi tiết các Inbound Rules:

- *Port 22 (SSH - Secure Shell):*
 - + Giao thức: TCP
 - + Nguồn cho phép: Có thể giới hạn theo địa chỉ IP cụ thể (ví dụ: chỉ cho phép IP của các thành viên nhóm) hoặc mở rộng cho tất cả (0.0.0.0/0) nếu cần truy cập từ nhiều vị trí khác nhau.
 - + Mục đích: Cho phép truy cập từ xa vào server thông qua SSH để quản trị, cấu hình, debug và maintain hệ thống.
 - + Đây là cổng duy nhất để quản trị viên có thể tương tác với server, thực hiện các lệnh command line, cài đặt phần mềm và kiểm tra log.
- *Port 80 (HTTP - HyperText Transfer Protocol):*
 - + Giao thức: TCP.
 - + Nguồn cho phép: 0.0.0.0/0 (mọi địa chỉ IP trên Internet).
 - + Mục đích: Cho phép người dùng cuối truy cập vào ứng dụng web thông qua trình duyệt mà không cần cài đặt gì thêm.
 - + Khi người dùng nhập địa chỉ IP của server vào trình duyệt, browser sẽ gửi HTTP request đến port 80, và frontend của hệ thống sẽ phản hồi bằng cách trả về giao diện web.

Mặc định, AWS cho phép tất cả traffic đi ra (all traffic, all protocols, all ports) từ instance. Điều này là cần thiết để server có thể kết nối ra ngoài Internet, ví dụ như kết nối đến MongoDB Atlas, hoặc tải các package từ npm registry, Docker Hub.

Nguyên tắc "Least Privilege" được áp dụng trong cấu hình Security Group: chỉ mở những cổng thực sự cần thiết cho hoạt động của hệ thống. Việc không mở các cổng không cần thiết (như port 3389 cho RDP, port 3306 cho MySQL trực tiếp) giúp giảm thiểu attack surface, giảm nguy cơ bị khai thác lỗ hổng bảo mật. Trong tương lai, nếu hệ thống cần triển khai HTTPS, nhóm sẽ cần mở thêm port 443 và cấu hình SSL certificate để mã hóa dữ liệu truyền tải.

3.1.3. Thiết lập IAM Role và Policy để kết nối dịch vụ

Để đảm bảo tính bảo mật và tuân thủ nguyên tắc "Least Privilege" (Đặc quyền tối thiểu), nhóm nghiên cứu không sử dụng Access Key/Secret Key lưu trong mã nguồn để kết nối các dịch vụ AWS. Thay vào đó, một IAM Role (Identity and Access Management) được khởi tạo và gán trực tiếp cho EC2 Instance.

Quy trình thực hiện bao gồm:

- Tạo IAM Role: Truy cập IAM Console, tạo một Role mới với loại thực thể tin cậy (Trusted Entity Type) là AWS Service và Use case là EC2.
- Gán quyền (Attach Policies): Nhóm đã gắn các Policy (chính sách) cần thiết để EC2 có thể tương tác với các dịch vụ khác:
 - + AmazonS3FullAccess: Cho phép ứng dụng Backend trên EC2 có quyền đọc/ghi dữ liệu hình ảnh vào S3 Bucket.
 - + CloudWatchAgentServerPolicy: Cho phép EC2 gửi log và metric (chỉ số) hệ thống lên CloudWatch.
 - + AmazonSSMManagedInstanceCore: Cho phép quản lý EC2 thông qua AWS Systems Manager mà không cần mở port SSH công khai.
- Gán Role cho EC2: Sau khi khởi tạo, Role này được gán vào EC2 Instance thông qua menu Actions > Security > Modify IAM role.

Việc sử dụng IAM Role giúp hệ thống tự động xác thực các yêu cầu API nội bộ trong mạng lưới AWS một cách an toàn và bảo mật hơn so với phương pháp truyền thống.

3.1.4. Cấu hình AWS Systems Manager (SSM) cho quản trị EC2

AWS Systems Manager (SSM) được triển khai để tăng cường khả năng quản trị và bảo mật cho máy chủ. Thay vì phụ thuộc hoàn toàn vào kết nối SSH (Port 22) vốn tiềm ẩn rủi ro bị quét port hoặc tấn công Brute-force, nhóm sử dụng Session Manager (một tính năng của SSM).

Quy trình cấu hình:

- Kiểm tra SSM Agent: Trên hệ điều hành Ubuntu Server (AMI mặc định), SSM Agent thường đã được cài đặt sẵn. Nhóm kiểm tra trạng thái hoạt động của agent bằng lệnh: `sudo systemctl status snap.amazon-ssm-agent.amazon-ssm-agent.service`.

- Kết nối Session Manager: Sau khi EC2 đã được gán IAM Role (có policy AmazonSSMManagedInstanceCore), server sẽ xuất hiện trong danh sách Managed Instances của Systems Manager.
- Vận hành: Quản trị viên có thể truy cập vào shell của server trực tiếp từ AWS Console thông qua trình duyệt web với kết nối được mã hóa hoàn toàn, giúp dễ dàng debug hoặc cấu hình server mà không cần quản lý SSH Key Pair phức tạp.

3.2. Chuẩn bị môi trường triển khai: Docker, Docker Compose và mã nguồn

```
PS C:\Users\Gigabyte\Downloads> ssh -i vinfastKey.pem ubuntu@54.84.69.254
The authenticity of host '54.84.69.254 (54.84.69.254)' can't be established.
ED25519 key fingerprint is SHA256:YZN4Hf3Au3scoeicLyN6gBWfp782WRB3Sw3fIZvicTA.
This host key is known by the following other names/addresses:
  C:\Users\Gigabyte/.ssh/known_hosts:9: 54.224.139.231
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.84.69.254' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jan 17 10:35:41 UTC 2026

System load:  1.42           Temperature:   -273.1 C
Usage of /:   23.4% of 28.02GB Processes:    171
Memory usage: 6%           Users logged in: 0
Swap usage:   0%           IPv4 address for enp39s0: 172.31.88.120

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

45 updates can be applied immediately.
5 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Mon Jan 12 06:44:42 2026 from 116.110.43.188
ubuntu@ip-172-31-88-120:~$
```

Hình 3.3. Kết nối SSH thành công vào EC2 instance

3.2.1. Cài đặt và cấu hình Docker Engine

```
ubuntu@ip-172-31-88-120:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
ubuntu@ip-172-31-88-120:~$ docker-compose --version
docker-compose version 1.29.2, build unknown
```

Hình 3.4. Docker và Docker Compose được cài đặt thành công trên EC2

Quy trình cài đặt Docker Engine trên Ubuntu Server: Docker Engine được cài đặt trực tiếp trên EC2 instance chạy hệ điều hành Ubuntu Server. Các bước chính bao gồm:

- Kết nối SSH vào EC2 instance bằng private key được tạo khi khởi tạo instance.
- Cập nhật hệ thống bằng cách cập nhật package index và các gói hiện có, nhằm đảm bảo tính tương thích và an toàn bảo mật cho môi trường cài đặt.
- Cài đặt các package cần thiết phục vụ cho việc cài Docker, bao gồm các công cụ hỗ trợ tải và xác thực package.
- Thêm GPG key và Docker repository chính thức, đảm bảo Docker được cài đặt từ nguồn đáng tin cậy.
- Cài đặt Docker Engine và các thành phần liên quan (Docker CLI, container runtime).
- Kiểm tra quá trình cài đặt, xác nhận Docker hoạt động đúng thông qua việc kiểm tra phiên bản và chạy container mẫu.
- Cấu hình quyền cho user, cho phép user ubuntu sử dụng Docker mà không cần quyền sudo, giúp thao tác thuận tiện hơn trong quá trình triển khai.
- Thiết lập Docker khởi động cùng hệ thống, đảm bảo Docker tự động chạy khi EC2 instance restart, góp phần tăng tính sẵn sàng cho hệ thống.

Việc sử dụng Docker giúp hệ thống tiếp cận theo hướng cloud-native, trong đó ứng dụng được đóng gói thành các container độc lập, dễ triển khai, dễ mở rộng và phù hợp với các nền tảng orchestration trên cloud như AWS ECS hoặc Kubernetes. Đây là một bước quan trọng trong việc áp dụng các nguyên tắc của điện toán đám mây vào dự án thực tế.

3.2.2. Cài đặt Docker Compose để quản lý multi-container

Docker Compose là công cụ hỗ trợ định nghĩa và vận hành nhiều Docker container đồng thời thông qua một file cấu hình duy nhất có định dạng YAML (`docker-compose.yml`). Trong khi Docker Engine quản lý từng container riêng lẻ, Docker Compose cho phép quản lý toàn bộ hệ thống ứng dụng như một tập hợp thống nhất, bao gồm các services, network và các cấu hình liên quan.

Đối với hệ thống Car-Management, ứng dụng bao gồm hai thành phần chính cần hoạt động đồng thời, đó là frontend (React application) và backend (Node.js/Express API). Nếu chỉ sử dụng Docker Engine, nhóm sẽ phải khởi chạy từng container bằng các lệnh `docker run` riêng biệt, gây khó khăn trong quản lý, dễ phát sinh lỗi cấu hình và khó tái sử dụng khi triển khai lại hệ thống.

Docker Compose giải quyết vấn đề này bằng cách cho phép ta định nghĩa toàn bộ các service trong file `docker-compose.yml`. Khi đó, chỉ cần sử dụng một lệnh duy nhất `docker-compose up`, toàn bộ hệ thống sẽ được khởi động một cách đồng bộ và nhất quán.

Quy trình cài đặt Docker Compose trên Ubuntu Server: Docker Compose được cài đặt trực tiếp trên EC2 instance chạy Ubuntu Server. Các bước chính bao gồm:

- Tải Docker Compose binary từ nguồn chính thức trên GitHub Releases, đảm bảo sử dụng phiên bản ổn định và tương thích với hệ thống.
- Cấp quyền thực thi cho file Docker Compose vừa tải, cho phép hệ thống có thể chạy công cụ này như một lệnh thông thường.
- Xác minh cài đặt, kiểm tra phiên bản Docker Compose để đảm bảo quá trình cài đặt thành công và sẵn sàng cho việc triển khai hệ thống.

3.2.3. Clone mã nguồn và cấu trúc project

```
ubuntu@ip-172-31-88-120:~$ git clone https://github.com/Harry1720/Car-Management.git
ls
fatal: destination path 'Car-Management' already exists and is not an empty directory.
Car-Management
```

Hình 3.5. Mã nguồn dự án được clone từ GitHub về EC2 instance

Sau khi môi trường triển khai với Docker và Docker Compose đã sẵn sàng, bước tiếp theo là đưa mã nguồn của ứng dụng lên server để tiến hành build và deploy. Nhóm sử dụng Git và GitHub để quản lý mã nguồn, đây là một best practice phổ biến trong phát triển phần mềm hiện đại.

Quy trình clone và thiết lập mã nguồn trên EC2: Quá trình đưa mã nguồn lên server được thực hiện theo các bước sau:

- Cài đặt Git trên EC2 instance để phục vụ việc quản lý và tải mã nguồn.
- Clone repository từ GitHub về server bằng lệnh clone, toàn bộ mã nguồn của dự án sẽ được tải về một thư mục trên EC2.
- Di chuyển vào thư mục project để tiếp tục các bước cấu hình và triển khai.

```
ubuntu@ip-172-31-88-120:~$ cd Car-Management/
ubuntu@ip-172-31-88-120:~/Car-Management$ ls be
Dockerfile  app.js  controllers  middlewares  node_modules  package.json  seed-cars.js  server.js
README.md   config  logs        models       package-lock.json  routes        seeds
ubuntu@ip-172-31-88-120:~/Car-Management$ ls car-management-react
Dockerfile  index.html  nginx.conf  package.json  src
eslint.config.js  netlify.toml  package-lock.json  public  vite.config.js
```

Hình 3.6. Cấu trúc project

Sau khi clone, mã nguồn dự án có cấu trúc chính như sau:

- *Thư mục be/*: Chứa toàn bộ logic phía server, được xây dựng bằng Node.js và Express framework. Backend chịu trách nhiệm xử lý các request từ frontend, thực hiện các thao tác CRUD (Create, Read, Update, Delete) với cơ sở dữ liệu, xác thực người dùng bằng JWT, và upload hình ảnh xe lên Cloudinary.
- *Thư mục car-management-react/*: Chứa ứng dụng phía client được xây dựng bằng React. Frontend đảm nhiệm việc hiển thị giao diện người dùng, gửi request đến backend API và render dữ liệu nhận được một cách động và tương tác.
- *File docker-compose.yml*: Là file cấu hình trung tâm của quá trình triển khai, định nghĩa các service frontend và backend, cách build container, cấu hình port, biến môi trường và network nội bộ để các service giao tiếp với nhau.

Khi tổ chức mã nguồn theo cấu trúc rõ ràng, tách biệt frontend và backend thì sẽ giúp hệ thống dễ bảo trì và phù hợp với tư duy kiến trúc cloud-native. Cách tổ chức này tạo nền tảng cho khả năng mở rộng trong tương lai, khi từng thành phần của hệ thống có thể được scale hoặc triển khai độc lập mà không ảnh hưởng đến các thành phần còn lại.

3.3. Cấu hình hệ thống và tích hợp Cloud Services

3.3.1. Cấu hình Backend thông qua biến môi trường (Environment Variables)

Trong quá trình triển khai hệ thống trên hạ tầng Cloud, nhóm phát triển đã áp dụng phương thức sử dụng Environment Variables (Biến môi trường) để cấu hình Backend. Đây là một trong những quy chuẩn kỹ thuật (Best Practice) quan trọng, đặc biệt đối với các ứng dụng triển khai trên môi trường Production. Thay vì nhúng trực tiếp (hard-code) các thông tin nhạy cảm vào mã nguồn, toàn bộ dữ liệu cấu hình được lưu trữ tập trung tại tệp tin `.env` và được nạp vào ứng dụng tại thời điểm thực thi (runtime). Phương pháp này mang lại các ưu điểm vượt trội sau:

- *Tính bảo mật*: Ngăn chặn việc lộ lọt các thông tin nhạy cảm (như mật khẩu cơ sở dữ liệu, API Keys) khi thực hiện commit mã nguồn lên các hệ thống quản lý phiên bản như GitHub.
- *Tính linh hoạt (Flexibility)*: Cho phép thay đổi thông số cấu hình giữa các môi trường (Development, Staging, Production) một cách nhanh chóng mà không cần tái cấu trúc hay sửa đổi mã nguồn.
- *Tách biệt mối quan tâm (Separation of Concerns)*: Phân tách độc lập giữa logic nghiệp vụ và cấu hình hệ thống, tuân thủ nguyên tắc 12-Factor App methodology trong phát triển ứng dụng Cloud-native.

Quy trình tạo và cấu hình file .env:

Bước 1: Trong thư mục backend, tạo file `.env`: `cd backend && nano .env`.

Bước 2: Nhập các biến môi trường cần thiết.

Bảng 3.1. Các biến môi trường

PORT	5000
MONGODB_URI	Chuỗi kết nối cơ sở dữ liệu
JWT_SECRET	Khóa bảo mật mã hóa
CLOUDINARY_CLOUD_NAME	Tên định danh Cloudinary
CLOUDINARY_API_KEY	Khóa API
CLOUDINARY_API_SECRET	Mã bảo mật API

Biến PORT xác định cổng dịch vụ mà Backend Server sử dụng để lắng nghe các yêu cầu HTTP. Hệ thống hiện đang cấu hình tại cổng 5000 (cổng mặc định phổ biến cho các ứng dụng Node.js). Khi triển khai bằng Docker, cổng này sẽ được ánh xạ (Port Mapping) thông qua Docker Compose, cho phép các yêu cầu từ Frontend hoặc người dùng bên ngoài truy cập vào dịch vụ.

Biến MONGODB_URI: Đây là chuỗi kết nối tới MongoDB Atlas – giải pháp Cơ sở dữ liệu dưới dạng dịch vụ (DBaaS) được quản lý hoàn toàn trên nền tảng đám mây. Chuỗi kết nối này bao gồm các thông số:

- Thông tin xác thực (Username, Password).
- Địa chỉ Cluster và tên Database.
- Các tham số cấu hình nâng cao (retryWrites, write concern).

Việc tích hợp MongoDB Atlas thay vì tự vận hành trên EC2 giúp hệ thống đạt được các chỉ số tối ưu về: Tự động sao lưu (Backups), Khả năng mở rộng (Scalability) và Tính sẵn sàng cao (High Availability) thông qua cơ chế Replica Sets.

Biến JWT_SECRET đóng vai trò là khóa bí mật phục vụ quy trình ký (sign) và xác thực (verify) JSON Web Tokens (JWT).

- Cơ chế: Khi người dùng xác thực thành công, hệ thống tạo ra một JWT chứa các thông tin định danh, được ký bởi khóa bí mật này. Client sẽ đính kèm token vào HTTP Header trong các yêu cầu tiếp theo để hệ thống kiểm tra quyền truy cập.
- Yêu cầu bảo mật: Khóa này được bảo mật tuyệt đối nhằm ngăn chặn các hình thức tấn công giả mạo danh tính (Identity Spoofing).

Các biến liên quan đến Cloudinary: Bao gồm CLOUDINARY_CLOUD_NAME, API_KEY và API_SECRET. Đây là các thông tin xác thực để kết nối với dịch vụ quản lý phương tiện truyền thông Cloudinary.

- Chức năng: Khi thực hiện tải lên hình ảnh, hệ thống sẽ truyền dữ liệu trực tiếp tới Cloudinary. Backend chỉ lưu trữ đường dẫn (URL) vào cơ sở dữ liệu.
- Lợi ích: Giải pháp này giúp tách biệt dữ liệu tĩnh (Static Assets) khỏi máy chủ EC2, hỗ trợ tối ưu hóa hình ảnh tự động và phân phối dữ liệu qua mạng CDN, giúp giảm tải tài nguyên server và tăng tốc độ trải nghiệm người dùng.

3.3.2. Tích hợp MongoDB Atlas - Database as a Service

MongoDB Atlas là dịch vụ cơ sở dữ liệu NoSQL được quản lý hoàn toàn trên nền tảng cloud. Nhóm lựa chọn MongoDB Atlas thay vì triển khai MongoDB trực tiếp trên EC2 instance nhằm tận dụng các lợi thế của Database-as-a-Service (DBaaS): quản lý tự động, khả năng mở rộng linh hoạt, và tách biệt tầng dữ liệu khỏi tầng ứng dụng theo mô hình kiến trúc cloud-native.

Quy trình thiết lập MongoDB Atlas: Tiến hành đăng ký tài khoản MongoDB Atlas và tạo cluster miễn phí tier M0 với cấu hình 512MB storage. Region được chọn là AWS Singapore (ap-southeast-1) nhằm tối ưu độ trễ kết nối từ Việt Nam. Sau khi cluster được khởi tạo, nhóm cấu hình Database Access bằng cách tạo database user với quyền readWrite, đồng thời thiết lập Network Access List để cho phép kết nối từ địa chỉ IP của EC2 instance.

Connection string được lấy từ Atlas Dashboard và cấu hình vào biến môi trường MONGODB_URI trong file .env của backend. Backend sử dụng thư viện Mongoose để thiết lập connection pool, cho phép tái sử dụng kết nối và tối ưu hiệu năng xử lý các thao tác CRUD với database.

Kiến trúc tách biệt database khỏi EC2 instance đảm bảo tính bền vững của dữ liệu (data persistence) khi server gặp sự cố hoặc cần restart, đồng thời tạo điều kiện thuận lợi cho việc mở rộng quy mô hệ thống trong tương lai.

3.3.3. Khởi tạo và tích hợp Amazon S3 - Object Storage

Để tối ưu hóa dung lượng lưu trữ trên EC2 và tăng tốc độ tải trang, hệ thống sử dụng Amazon S3 (Simple Storage Service) để lưu trữ toàn bộ hình ảnh xe của Showroom.

Quy trình triển khai S3:

- Khởi tạo Bucket:
 - + Tạo một S3 Bucket mới với tên định danh duy nhất (ví dụ: vinfast-showroom-images).
 - + Chọn Region trùng với Region của EC2 (ví dụ: ap-southeast-1 - Singapore) để giảm độ trễ (latency) khi upload và truy xuất ảnh.
- Cấu hình quyền truy cập (Permissions):
 - + Tắt chế độ Block all public access (hoặc cấu hình Bucket Policy) để cho phép người dùng cuối có thể xem được ảnh xe thông qua đường dẫn URL.
 - + Thiết lập CORS (Cross-Origin Resource Sharing) cho phép Frontend (từ domain/IP của ứng dụng) có thể fetch tài nguyên hình ảnh.
- Tích hợp vào ứng dụng:
 - + Backend sử dụng thư viện aws-sdk để giao tiếp với S3.
 - + Khi quản trị viên upload ảnh xe, Backend sẽ stream dữ liệu trực tiếp lên S3 Bucket và nhận về URL của ảnh. URL này sau đó được lưu vào MongoDB để phục vụ hiển thị.

3.3.4. Thiết lập giám sát hệ thống với Amazon CloudWatch

Để đảm bảo hệ thống hoạt động ổn định và phát hiện sớm các sự cố quá tải, nhóm đã tích hợp dịch vụ giám sát Amazon CloudWatch.

Quy trình thiết lập giám sát:

- Kích hoạt Monitoring: Trong quá trình cấu hình EC2, nhóm kích hoạt tính năng Detailed Monitoring để thu thập dữ liệu với tần suất 1 phút/lần (thay vì 5 phút mặc định).
- Theo dõi Dashboard: Một Dashboard tùy chỉnh được tạo trên CloudWatch để hiển thị trực quan các chỉ số quan trọng (Metrics):
 - + CPU Utilization: Giám sát mức độ sử dụng CPU để phát hiện tình trạng quá tải xử lý.

- + Network In/Out: Theo dõi lưu lượng mạng truy cập vào website.
- + Status Check Failed: Cảnh báo nếu phần cứng hoặc phần mềm của Instance gặp sự cố nghiêm trọng.
- Thiết lập Alarm (Cảnh báo): Cấu hình CloudWatch Alarm để gửi thông báo nếu mức sử dụng CPU vượt quá 80% trong 5 phút liên tục, giúp quản trị viên kịp thời thực hiện các biện pháp mở rộng (Scaling) hoặc xử lý sự cố.

3.4. Build và triển khai hệ thống với Docker Compose

Sau khi hoàn tất các bước cấu hình hạ tầng và môi trường, bước tiếp theo là build Docker images và triển khai toàn bộ hệ thống bằng Docker Compose.

3.4.1. Phân tích file *docker-compose.yml*

File *docker-compose.yml* đóng vai trò là bản mô tả tổng thể của hệ thống, cho phép định nghĩa các service cần triển khai, cách build container, cấu hình network nội bộ và mối quan hệ phụ thuộc giữa các service. Cấu trúc cơ bản của file này trong dự án như sau:

```
version: "3.8"

services:
  backend:
    build: ./be
    container_name: vinfast-backend
    ports:
      - "5000:5000"
    env_file:
      - ./be/.env
    restart: always
  frontend:
    build:
      context: ./car-management-react
    args:
      VITE_API_BASE_URL: http://54.224.139.231:5000/api
    container_name: vinfast-frontend
```

```
ports:
  - "80:80"

depends_on:
  - backend

restart: always
```

Giải thích các thành phần chính

- version: '3.8': Chỉ định phiên bản Docker Compose file format, đảm bảo tương thích với Docker Engine hiện đại.
- Services: Định nghĩa các container cần chạy trong hệ thống, bao gồm backend và frontend.

Backend service:

- build: Chỉ định thư mục chứa Dockerfile để build backend image.
- container_name: Đặt tên container rõ ràng để dễ quản lý và debug.
- ports: Ánh xạ port 5000 của container ra port 5000 của host.
- env_file: Load các biến môi trường từ file .env.
- restart: always: Đảm bảo container tự động restart khi gặp sự cố hoặc khi Docker daemon restart.

Frontend service:

- build: Build image frontend từ source React.
- ports: Map port 80 để người dùng truy cập qua trình duyệt.
- depends_on: Đảm bảo backend được khởi động trước frontend.

Docker Compose tự động tạo một network bridge nội bộ, cho phép các service giao tiếp với nhau thông qua tên service, mô phỏng cơ chế service discovery trong các hệ thống cloud.

3.4.2. Dockerfile của Backend

Dockerfile backend định nghĩa quy trình build image cho server-side application. Nội dung Dockerfile như sau:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Dockerfile sử dụng image Node.js 18 trên Alpine Linux nhằm giảm kích thước image. Việc copy package.json trước giúp tận dụng Docker layer caching, từ đó tăng tốc độ build khi source code không thay đổi.

3.4.3. Dockerfile của Frontend và vai trò của Nginx

Frontend sử dụng kỹ thuật multi-stage build, một best practice trong triển khai ứng dụng frontend.

```
# Build stage
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Serve stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Ở stage đầu tiên, React application được build thành các static files tối ưu trong thư mục dist. Stage thứ hai chỉ sử dụng Nginx để serve các static files này, giúp image production nhỏ gọn và bảo mật hơn.

Vai trò của Nginx trong hệ thống

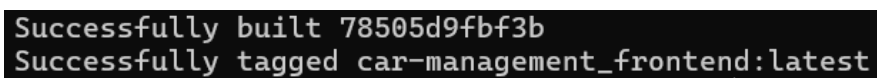
- Serve frontend application: Nginx phục vụ các static files (HTML, CSS, JavaScript) được build từ React, giúp frontend hoạt động hiệu quả và ổn định.
- Hiệu năng cao: Nginx là web server nhẹ, tối ưu cho việc phục vụ nội dung tĩnh, phù hợp với môi trường cloud.
- Tách biệt frontend và backend: Frontend được triển khai độc lập với backend, đúng với tư duy kiến trúc cloud-native.
- Sẵn sàng cho mở rộng: Trong các kiến trúc nâng cao, Nginx có thể được sử dụng làm reverse proxy hoặc load balancer.

Việc sử dụng Nginx thay vì chạy React bằng Node.js trong production giúp giảm tải tài nguyên và tăng tính ổn định cho hệ thống.

3.4.4. Quy trình build và deploy hệ thống

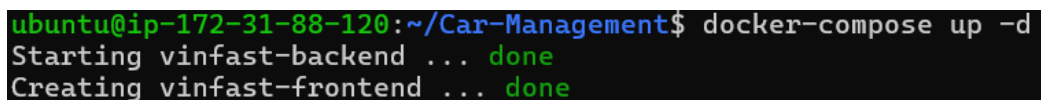
Quá trình triển khai hệ thống được thực hiện bằng Docker Compose với các bước chính:

- Truy cập EC2 instance và di chuyển vào thư mục project.
- Kiểm tra file .env của backend để đảm bảo các biến môi trường đã được cấu hình đúng.
- Build image và khởi chạy các container bằng lệnh: `docker-compose up -d --build`.



```
Successfully built 78505d9fbf3b
Successfully tagged car-management_frontend:latest
```

Hình 3.7. Quá trình build Docker images cho frontend và backend



```
ubuntu@ip-172-31-88-120:~/Car-Management$ docker-compose up -d
Starting vinfast-backend ... done
Creating vinfast-frontend ... done
```

Hình 3.8. Các container được khởi chạy bằng Docker Compose

Docker Compose sẽ tự động:

- Tạo network nội bộ
- Build image cho backend và frontend
- Tạo và khởi chạy các container theo đúng thứ tự phụ thuộc

Sau khi deploy, nhóm kiểm tra trạng thái các container và logs để đảm bảo hệ thống hoạt động ổn định.

3.5. Kiểm thử và xác minh hệ thống sau triển khai

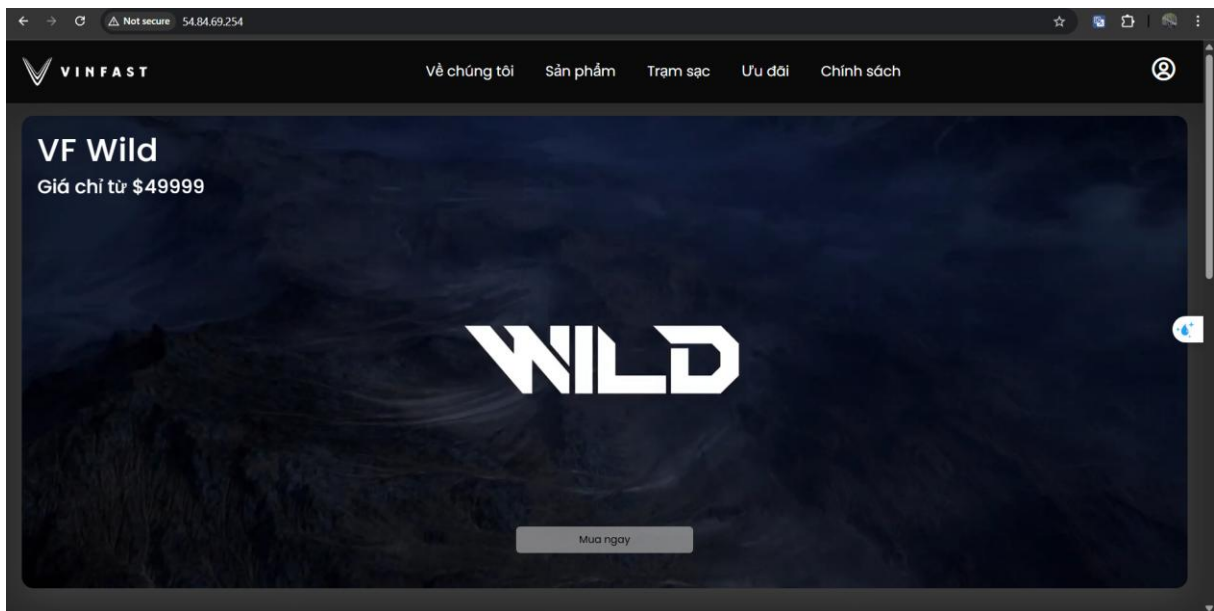
```
ubuntu@ip-172-31-88-120:~/Car-Management$ docker-compose ps
```

Name	Command	State	Ports
vinfast-backend	docker-entrypoint.sh node ...	Up	0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
vinfast-frontend	/docker-entrypoint.sh nginx ...	Up	0.0.0.0:80->80/tcp, :::80->80/tcp

Hình 3.9. Trạng thái các container sau khi triển khai

Sau khi hệ thống được triển khai thành công trên nền tảng AWS, nhóm tiến hành kiểm thử toàn diện nhằm xác minh rằng các chức năng của hệ thống hoạt động đúng yêu cầu trong môi trường production. Quá trình kiểm thử tập trung vào khả năng truy cập, tính đúng đắn của chức năng nghiệp vụ, tích hợp các dịch vụ cloud, cũng như tính ổn định và hiệu năng của hệ thống.

3.5.1. Kiểm tra khả năng truy cập và giao diện người dùng



Hình 3.10. Giao diện hệ thống quản lý xe

Nhóm kiểm tra khả năng truy cập hệ thống thông qua trình duyệt web bằng cách sử dụng Public IPv4 của EC2 instance. Kết quả cho thấy giao diện frontend được hiển thị chính xác, bao gồm các thành phần chính như trang chủ, thanh điều hướng và các trang chức năng.

Ngoài ra, giao diện người dùng được kiểm tra trên nhiều kích thước màn hình khác nhau (desktop, tablet và mobile) nhằm đảm bảo khả năng hiển thị phù hợp và đáp ứng yêu cầu responsive design.

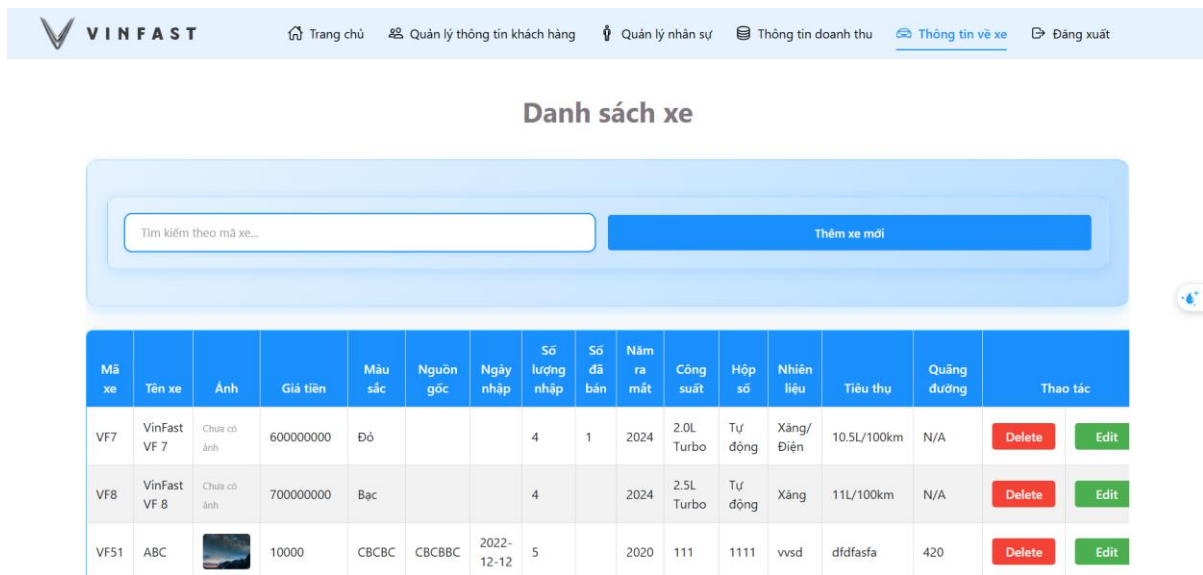
3.5.2. Kiểm tra chức năng xác thực và phân quyền quản lý

Hệ thống được thiết kế theo mô hình admin-only, do đó chỉ các tài khoản quản lý được cấp sẵn trong hệ thống mới có quyền truy cập.

Kết quả kiểm thử cho thấy:

- Người quản lý có thể đăng nhập thành công bằng thông tin tài khoản hợp lệ.
- Backend xác thực thông tin đăng nhập và tạo JSON Web Token (JWT) để quản lý phiên làm việc.
- Các trang quản trị (dashboard, quản lý xe) không thể truy cập nếu chưa đăng nhập, đảm bảo an toàn cho hệ thống quản lý nội bộ.

3.5.3. Kiểm tra chức năng quản lý xe (CRUD)



Mã xe	Tên xe	Ảnh	Giá tiền	Màu sắc	Nguồn gốc	Ngày nhập	Số lượng nhập	Số đã bán	Năm ra mắt	Công suất	Hộp số	Nhiên liệu	Tiêu thụ	Quãng đường	Thao tác
VF7	VinFast VF 7	Chưa có ảnh	600000000	Đỏ			4	1	2024	2.0L Turbo	Tự động	Xăng/Điện	10.5L/100km	N/A	Delete Edit
VF8	VinFast VF 8	Chưa có ảnh	700000000	Bạc			4		2024	2.5L Turbo	Tự động	Xăng	11L/100km	N/A	Delete Edit
VF51	ABC		10000	CBCBC	CBCBBC	2022-12-12	5		2020	111	1111	vvsd	dfdfasfa	420	Delete Edit

Hình 3.11. Giao diện quản lý xe sau khi admin đăng nhập

Sau khi đăng nhập với tài khoản quản lý, các chức năng quản lý xe được kiểm thử bao gồm:

- Thêm xe: Quản lý có thể thêm xe mới kèm hình ảnh. Hình ảnh được upload lên Cloudinary và URL được lưu trong MongoDB Atlas.
- Xem danh sách xe: Hệ thống hiển thị danh sách xe hiện có, dữ liệu được lấy từ backend API.
- Cập nhật thông tin xe: Quản lý có thể chỉnh sửa thông tin xe và dữ liệu được cập nhật chính xác trong database.

- Xóa xe: Quản lý có thể xóa xe khỏi hệ thống, dữ liệu được loại bỏ khỏi database.

Các chức năng trên hoạt động ổn định và đúng với yêu cầu của hệ thống quản lý nội bộ.

3.5.4. Kiểm tra tính ổn định và khả năng phục hồi

Nhóm tiến hành kiểm thử bằng cách khởi động lại container backend và EC2 instance. Kết quả cho thấy hệ thống tự động khởi động lại các container nhờ cấu hình Docker Compose, đồng thời dữ liệu quản lý xe và hình ảnh không bị ảnh hưởng do được lưu trữ trên các dịch vụ cloud bên ngoài.

3.5.5. Kết quả kiểm thử tổng hợp

Kết quả kiểm thử cho thấy hệ thống quản lý xe dành cho quản trị viên hoạt động ổn định và đáp ứng đúng các yêu cầu đề ra. Chỉ các tài khoản có quyền quản lý mới được phép truy cập và thực hiện các thao tác trên dữ liệu, đảm bảo cơ chế phân quyền hoạt động chính xác. Các chức năng quản lý xe được triển khai đầy đủ và vận hành đúng theo thiết kế, đồng thời dữ liệu được đảm bảo an toàn và không phụ thuộc trực tiếp vào EC2 instance nhờ việc lưu trữ trên các dịch vụ cloud bên ngoài.

CHƯƠNG 4.

VẬN HÀNH, BẢO MẬT & KHẢ NĂNG MỞ RỘNG HỆ THỐNG

4.1. Cơ chế vận hành hệ thống

Hệ thống Showroom ô tô VinFast được vận hành trên hạ tầng máy chủ ảo Amazon EC2 (Ubuntu Server). Cơ chế vận hành cốt lõi dựa trên nền tảng container hóa với Docker và công cụ điều phối Docker Compose. Trong file cấu hình docker-compose.yml, các container Frontend (React/Nginx) và Backend (Node.js) được định nghĩa như các dịch vụ (services) hoạt động song song. Điểm đặc biệt trong cơ chế vận hành là việc áp dụng cấu hình restart: always cho các container. Cơ chế này đảm bảo tính sẵn sàng cao (High Availability): nếu ứng dụng gặp lỗi crash hoặc máy chủ EC2 khởi động lại, Docker daemon sẽ tự động kích hoạt lại các container mà không cần sự can thiệp thủ công của quản trị viên.

4.2. Cập nhật & quản lý phiên bản

Quy trình cập nhật và bảo trì hệ thống được thực hiện nhất quán thông qua GitHub kết hợp với cơ chế build của Docker, nhằm đảm bảo phiên bản triển khai trên cloud luôn đồng bộ với mã nguồn phát triển. Mã nguồn của hệ thống được quản lý tập trung trên GitHub, sau đó được cập nhật về EC2 instance thông qua lệnh git pull. Sau khi mã nguồn mới được tải về, lệnh docker-compose up -d --build được thực thi để tái xây dựng các Docker image và khởi động lại các container với phiên bản code mới. Quy trình này giúp đảm bảo tính nhất quán giữa các môi trường, hạn chế sai lệch trong quá trình vận hành và hỗ trợ bảo trì hệ thống hiệu quả.

4.3. Quản lý cấu hình

Hệ thống được thiết kế và triển khai theo nguyên tắc 12-Factor App dành cho các ứng dụng cloud-native, trong đó phần cấu hình (configuration) được tách biệt hoàn toàn khỏi mã nguồn (source code). Cách tiếp cận này giúp tăng tính bảo mật, khả năng mở rộng và thuận tiện trong quá trình vận hành trên nền tảng điện toán đám mây.

Các tham số cấu hình và thông tin nhạy cảm của hệ thống như cổng dịch vụ (PORT), chuỗi kết nối cơ sở dữ liệu (MONGODB_URI), khóa bí mật phục vụ xác thực (JWT_SECRET) và thông tin cấu hình liên quan đến dịch vụ lưu trữ đối tượng Amazon S3 không được lưu cứng (hard-code) trong mã nguồn ứng dụng. Thay vào đó, các giá

trị này được quản lý tập trung thông qua biến môi trường trong tệp tin `.env` và được nạp vào môi trường chạy của ứng dụng tại thời điểm runtime.

Đối với việc truy cập dịch vụ Amazon S3, hệ thống không sử dụng AWS Access Key và Secret Key trong cấu hình ứng dụng. Thay vào đó, máy ảo EC2 được gán IAM Role phù hợp, cho phép ứng dụng backend truy cập và thao tác với S3 thông qua cơ chế phân quyền nội bộ của AWS. Cách tiếp cận này giúp loại bỏ việc quản lý khóa truy cập thủ công, giảm thiểu rủi ro rò rỉ thông tin xác thực và tuân thủ các best practice về bảo mật của AWS.

Việc quản lý cấu hình theo hướng tách biệt và sử dụng IAM Role góp phần nâng cao mức độ an toàn, tính linh hoạt và khả năng mở rộng của hệ thống trong quá trình triển khai và vận hành thực tế.

4.4. Giám sát hệ thống

Để đảm bảo hệ thống hoạt động ổn định, nhóm thực hiện giám sát bằng các công cụ sẵn có của Docker. Cụ thể, lệnh `docker logs` được sử dụng để theo dõi nhật ký hoạt động của các container frontend và backend, qua đó phát hiện các lỗi trong quá trình chạy và kiểm tra các yêu cầu HTTP gửi đến hệ thống. Đồng thời, lệnh `docker stats` được dùng để giám sát mức tiêu thụ tài nguyên như CPU, RAM và lưu lượng mạng của từng container, giúp đảm bảo EC2 instance không bị quá tải. Ngoài ra, trạng thái hoạt động của các dịch vụ cũng được kiểm tra thông qua lệnh `docker-compose ps` nhằm xác nhận các container đang chạy ổn định và lắng nghe đúng cổng đã cấu hình.

Bên cạnh việc giám sát ở mức container thông qua các công cụ của Docker, hệ thống còn sử dụng dịch vụ Amazon CloudWatch để theo dõi và đánh giá tình trạng hoạt động của máy ảo EC2 ở mức hạ tầng. Amazon CloudWatch cung cấp các chỉ số (metrics) theo thời gian thực như mức sử dụng CPU, dung lượng bộ nhớ, lưu lượng mạng vào/ra và trạng thái của máy ảo, giúp nhóm nắm bắt tổng thể hiệu năng của hệ thống trong quá trình vận hành. Thông qua CloudWatch, các chỉ số quan trọng của EC2 được thu thập và hiển thị tập trung trên giao diện quản lý AWS, cho phép phát hiện sớm các dấu hiệu quá tải hoặc bất thường ở tầng hạ tầng, ngay cả khi container vẫn đang hoạt động. Điều này đặc biệt quan trọng trong bối cảnh hệ thống được triển khai trên máy ảo dùng chung cho nhiều dịch vụ frontend và backend.

Việc kết hợp giám sát ở hai cấp độ, bao gồm mức container (Docker) và mức hạ tầng (Amazon CloudWatch), giúp hệ thống đạt được khả năng quan sát (observability) toàn diện. Cách tiếp cận này hỗ trợ hiệu quả cho công tác vận hành, xử lý sự cố và đảm bảo hệ thống hoạt động ổn định, liên tục trong môi trường điện toán đám mây.

4.5. Khôi phục

Kiến trúc hệ thống được thiết kế nhằm đảm bảo tính bền vững của dữ liệu ngay cả khi máy chủ tính toán EC2 gặp sự cố. Toàn bộ dữ liệu nghiệp vụ được lưu trữ trên MongoDB Atlas dưới dạng dịch vụ cơ sở dữ liệu độc lập, tách biệt hoàn toàn khỏi vòng đời của container và EC2 instance, trong khi dữ liệu đa phương tiện như hình ảnh được lưu trữ trên dịch vụ AWS S3. Nhờ cách tiếp cận này, việc khởi động lại hoặc thay thế máy chủ EC2 không gây mất mát dữ liệu, giúp hệ thống duy trì khả năng phục hồi và nhanh chóng khôi phục trạng thái sau sự cố.

4.6. Bảo mật

Hệ thống áp dụng mô hình bảo mật đa lớp (Defense in Depth) nhằm bảo vệ toàn diện ứng dụng và dữ liệu trong quá trình triển khai và vận hành trên nền tảng điện toán đám mây.

Ở tầng hạ tầng, AWS Security Group được sử dụng như một tường lửa ảo, chỉ cho phép truy cập thông qua các cổng thiết yếu như cổng 22 (SSH) phục vụ quản trị và cổng 80 cho dịch vụ web, đồng thời chặn các truy cập không hợp lệ khác từ bên ngoài. Cơ chế này giúp kiểm soát lưu lượng mạng vào/ra máy ảo EC2 một cách hiệu quả.

Ở tầng quản trị và vận hành, hệ thống sử dụng AWS Systems Manager, đặc biệt là tính năng Session Manager, để quản lý và truy cập máy ảo EC2 mà không cần sử dụng trực tiếp SSH và private key. Việc loại bỏ truy cập SSH truyền thống giúp giảm thiểu rủi ro liên quan đến việc lộ hoặc quản lý khóa truy cập, đồng thời tăng cường khả năng kiểm soát và ghi nhận hoạt động quản trị thông qua nhật ký truy cập tập trung.

Ở tầng phân quyền và xác thực dịch vụ, IAM Role được gắn trực tiếp vào máy ảo EC2 nhằm kiểm soát quyền truy cập của hệ thống đối với các dịch vụ AWS khác. Thay vì lưu trữ AWS Access Key trong mã nguồn hoặc tệp cấu hình, EC2 sử dụng IAM Role để assume quyền truy cập một cách an toàn, tuân thủ các best practice về bảo mật của AWS. Cơ chế này giúp hạn chế nguy cơ rò rỉ thông tin xác thực và đơn giản hóa việc quản lý quyền trong hệ thống.

Ở tầng ứng dụng, cơ chế xác thực JWT (JSON Web Token) được triển khai, trong đó token được ký bằng khóa bí mật để xác thực người dùng theo mô hình stateless. Cách tiếp cận này không yêu cầu lưu trữ session trên server, phù hợp với kiến trúc phân tán và hỗ trợ khả năng mở rộng hệ thống.

Ở tầng dữ liệu, cơ sở dữ liệu MongoDB Atlas không được công khai trực tiếp ra Internet mà chỉ cho phép kết nối từ các địa chỉ IP được cấp phép thông qua cơ chế IP Whitelist. Biện pháp này giúp giảm thiểu đáng kể bề mặt tấn công và bảo vệ dữ liệu khỏi các truy cập trái phép.

Việc kết hợp các cơ chế bảo mật ở nhiều tầng khác nhau, từ hạ tầng, quản trị, phân quyền cho đến ứng dụng và dữ liệu, giúp hệ thống đạt được mức độ an toàn cao, phù hợp với yêu cầu triển khai ứng dụng trong môi trường điện toán đám mây.

CHƯƠNG 5.

ĐÁNH GIÁ KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN

5.1. Đánh giá kết quả đạt được

Sau quá trình nghiên cứu và thực hiện, nhóm đã hoàn thành việc triển khai hệ thống website showroom ô tô VinFast lên nền tảng điện toán đám mây AWS. Các kết quả đạt được cụ thể như sau:

Về hạ tầng: Đã khởi tạo thành công máy chủ ảo EC2 (Instance type: m7i-flex.large) chạy hệ điều hành Ubuntu Server 22.04, đảm bảo tài nguyên tính toán cho hệ thống.

Về công nghệ triển khai: Đã áp dụng thành công công nghệ Containerization (Docker) và công cụ điều phối Docker Compose để đóng gói và vận hành ứng dụng. Hệ thống hoạt động nhất quán giữa môi trường development và production.

Về kiến trúc hệ thống: Đã thực hiện tách biệt hoàn toàn Frontend và Backend, đồng thời tích hợp thành công các dịch vụ Cloud bên thứ ba (MongoDB Atlas cho cơ sở dữ liệu và Cloudinary cho lưu trữ hình ảnh).

Về chức năng: Hệ thống trên Cloud hoạt động ổn định, đảm bảo các chức năng quản trị (Admin) như đăng nhập, quản lý xe (CRUD), và upload hình ảnh hoạt động chính xác thông qua Public IP của AWS.

5.2. Ưu điểm

Giải pháp triển khai hiện tại mang lại nhiều ưu điểm rõ rệt về cả mặt kỹ thuật và vận hành. Việc tách biệt cơ sở dữ liệu MongoDB Atlas và hệ thống lưu trữ đối tượng Amazon S3 giúp giảm tải đáng kể cho máy chủ EC2, đồng thời tận dụng khả năng mở rộng linh hoạt và độ bền dữ liệu cao của S3 so với lưu trữ cục bộ. Về bảo mật, hệ thống áp dụng IAM Role thay cho việc sử dụng Access Key/Secret Key trong mã nguồn, qua đó loại bỏ nguy cơ lộ thông tin xác thực khi quản lý và chia sẻ code. Khả năng giám sát và vận hành được nâng cao thông qua Amazon CloudWatch, cho phép theo dõi trạng thái và hiệu năng hệ thống theo thời gian thực, kết hợp với AWS Systems Manager để quản trị và cấu hình server an toàn mà không cần sử dụng SSH Key Pair. Bên cạnh đó, việc container hóa ứng dụng bằng Docker giúp chuẩn hóa môi trường chạy, cho phép

triển khai hoặc khôi phục toàn bộ hệ thống một cách nhanh chóng và nhất quán chỉ với một lệnh duy nhất.

5.3. Hạn chế

Bên cạnh những kết quả đạt được, hệ thống triển khai hiện tại vẫn còn tồn tại một số hạn chế cần được khắc phục trong các giai đoạn phát triển tiếp theo. Cụ thể, địa chỉ IP Public của EC2 đang bị gán cứng vào biến môi trường trong quá trình build Docker Image, gây khó khăn khi IP thay đổi hoặc khi mở rộng hệ thống sang nhiều máy chủ. Về mặt bảo mật mạng, mặc dù đã sử dụng Security Group, cổng API Backend vẫn đang mở cho toàn bộ Internet, tiềm ẩn nguy cơ bị tấn công trực tiếp vào hệ thống. Ngoài ra, hệ thống hiện chỉ hoạt động trên HTTP, chưa triển khai HTTPS nên mức độ an toàn của dữ liệu truyền tải, đặc biệt là thông tin đăng nhập và JWT token, chưa được đảm bảo. Quy trình triển khai cũng còn mang tính thủ công, yêu cầu truy cập trực tiếp vào server để cập nhật và build lại ứng dụng, chưa có cơ chế CI/CD tự động. Cuối cùng, hệ thống mới chỉ hỗ trợ mở rộng theo chiều dọc trên một EC2 duy nhất, do đó có nguy cơ trở thành điểm nghẽn hiệu năng khi số lượng người dùng và lưu lượng truy cập tăng cao.

5.4. Hướng phát triển

Dựa trên các phân tích về ưu điểm và hạn chế, nhóm đề xuất một số hướng phát triển trong giai đoạn tiếp theo nhằm nâng cao tính an toàn, khả năng vận hành và mở rộng của hệ thống. Trước hết, cần nâng cấp hạ tầng mạng và bảo mật bằng cách đăng ký tên miền riêng, sử dụng AWS Route 53 để quản lý DNS, đồng thời triển khai HTTPS thông qua AWS Certificate Manager (ACM) kết hợp với Load Balancer hoặc CloudFront nhằm đảm bảo mã hóa dữ liệu truyền tải. Song song với đó, cấu hình lại mạng và Security Group để Backend chỉ tiếp nhận request từ Frontend hoặc qua mạng nội bộ, hạn chế việc mở trực tiếp cổng API ra Internet.

Tiếp theo, hệ thống nên được tự động hóa quy trình triển khai (CI/CD) bằng cách xây dựng pipeline với GitHub Actions hoặc AWS CodePipeline, cho phép tự động build Docker Image, đẩy image lên Amazon ECR và cập nhật ứng dụng trên EC2 mỗi khi có thay đổi mã nguồn, qua đó giảm phụ thuộc vào thao tác thủ công và hạn chế lỗi vận hành. Bên cạnh đó, việc tối ưu hóa phân phối nội dung có thể được thực hiện bằng cách tích hợp Amazon CloudFront (CDN) phía trước Amazon S3, giúp tăng tốc độ tải hình ảnh cho người dùng ở nhiều khu vực địa lý khác nhau và giảm chi phí băng thông. Cuối

cùng, để đáp ứng nhu cầu tăng trưởng, hệ thống cần hướng tới mở rộng quy mô linh hoạt bằng cách chuyển từ mô hình Docker Compose đơn lẻ sang các nền tảng quản lý container như AWS ECS hoặc EKS, kết hợp với Application Load Balancer (ALB) và Auto Scaling nhằm tự động điều chỉnh tài nguyên theo lưu lượng truy cập thực tế.

KẾT LUẬN

Đề tài “Triển khai website showroom ô tô Vinfast trên nền tảng điện toán đám mây AWS” đã được thực hiện và hoàn thành với những kết quả đáng khích lệ. Thông qua quá trình nghiên cứu, thiết kế và triển khai, một hệ thống web hoàn chỉnh đã được xây dựng thành công, đáp ứng được các mục tiêu đề ra ban đầu về mặt chức năng, hiệu năng và khả năng triển khai trên nền tảng cloud.

Về mặt công nghệ, dự án đã áp dụng thành công kiến trúc phân tán hiện đại với frontend được xây dựng bằng ReactJS và backend sử dụng Node.js/Express.js, giao tiếp qua RESTful API. Việc áp dụng các dịch vụ Cloud native của AWS, đặc biệt là Amazon S3 cho lưu trữ hình ảnh và Docker/Docker Compose cho container hóa, đã giúp tối ưu tài nguyên, chuẩn hóa môi trường vận hành và đảm bảo tính nhất quán từ giai đoạn phát triển đến triển khai thực tế.

Về chức năng, hệ thống đã cung cấp đầy đủ các tính năng cần thiết cho cả hai nhóm người dùng chính. Đối với khách hàng, họ có thể dễ dàng xem danh sách các dòng xe, tìm kiếm và xem chi tiết thông tin sản phẩm với hình ảnh chất lượng cao, sau đó thực hiện đặt cọc trực tuyến một cách thuận tiện. Đối với quản trị viên, hệ thống cung cấp một dashboard tổng quan với các biểu đồ thống kê trực quan và các module quản lý đầy đủ cho xe, khách hàng, đặt cọc, giao dịch, kế toán và nhân sự. Cơ chế xác thực JWT và phân quyền đã được triển khai để đảm bảo an toàn thông tin và phân quyền truy cập hợp lý. Giao diện người dùng được thiết kế responsive, tương thích tốt trên nhiều thiết bị từ desktop đến mobile, mang lại trải nghiệm sử dụng mượt mà và chuyên nghiệp.

Về triển khai và vận hành, dự án đã tiếp cận theo hướng chuyên nghiệp khi sử dụng EC2, IAM Role, AWS Systems Manager, CloudWatch và Nginx Reverse Proxy, từng bước tiệm cận mô hình vận hành thực tế trên AWS. Điều này giúp nhóm tích lũy kinh nghiệm thực tiễn trong việc quản trị hạ tầng cloud, bảo mật và giám sát hệ thống.

Qua quá trình thực hiện đề tài, nhiều kiến thức và kỹ năng quý báu đã được tích lũy. Về mặt kỹ thuật, người thực hiện đã có cơ hội làm việc với các công nghệ hiện đại, kinh nghiệm về thiết kế RESTful API, xây dựng kiến trúc microservices, xử lý bảo mật web application, và quản lý deployment pipeline đã được trau dồi thông qua thực hành. Ngoài ra, việc làm việc với các dịch vụ cloud như MongoDB Atlas đã giúp hiểu rõ hơn

về ưu điểm của các managed services trong việc giảm tải công việc vận hành và tăng tính sẵn sàng của hệ thống.

Tuy nhiên, do phạm vi thời gian và nguồn lực của một đề tài học tập, hệ thống vẫn còn một số hạn chế cần được cải thiện. Thứ nhất, hệ thống chưa tích hợp payment gateway thực tế để xử lý thanh toán trực tuyến, hiện tại chỉ dừng ở bước đặt cọc và ghi nhận giao dịch thủ công. Thứ hai, chưa triển khai HTTPS/SSL certificate cho kết nối bảo mật, đây là yêu cầu cần thiết cho ứng dụng production xử lý thông tin người dùng. Thứ ba, chưa có CI/CD pipeline tự động để streamline quy trình build, test và deploy. Thứ tư, hiệu năng của hệ thống chưa được tối ưu hóa tối đa với các kỹ thuật như caching, database indexing, hoặc CDN cho static assets. Những hạn chế này có thể được khắc phục trong các giai đoạn phát triển tiếp theo khi hệ thống được đưa vào vận hành thực tế.

Tổng kết lại, đề tài đã hoàn thành tốt mục tiêu xây dựng và triển khai một hệ thống web hiện đại trên nền tảng AWS, thể hiện khả năng vận dụng kiến thức công nghệ thông tin vào giải quyết bài toán thực tế trong lĩnh vực thương mại điện tử ô tô. Những kết quả đạt được tạo nền tảng vững chắc để phát triển hệ thống theo hướng SaaS trong tương lai, đồng thời là cơ hội quý báu giúp nhóm rèn luyện tư duy hệ thống, kỹ năng triển khai và tác phong làm việc chuyên nghiệp. Quan trọng hơn, việc hoàn thành đề tài không chỉ là cột mốc kết thúc một môn học mà còn là cơ hội để chúng em cọ xát với môi trường phát triển dự án thực tế. Đối mặt với các vấn đề phức tạp xuyên suốt từ hệ thống đến giao diện đã giúp chúng em rèn luyện tư duy giải quyết vấn đề và tác phong làm việc bài bản. Chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Hàn Trung Định đã định hướng & hướng dẫn tận tình, cùng các Thầy Cô Viện Công nghệ thông tin và Điện, Điện tử đã xây dựng nền tảng tri thức vững chãi, giúp chúng em có đủ tự tin để hoàn thành dự án này & chinh phục những thử thách sắp tới.

PHỤ LỤC

- Link GitHub mã nguồn của dự án: <https://github.com/Harry1720/Car-Management>